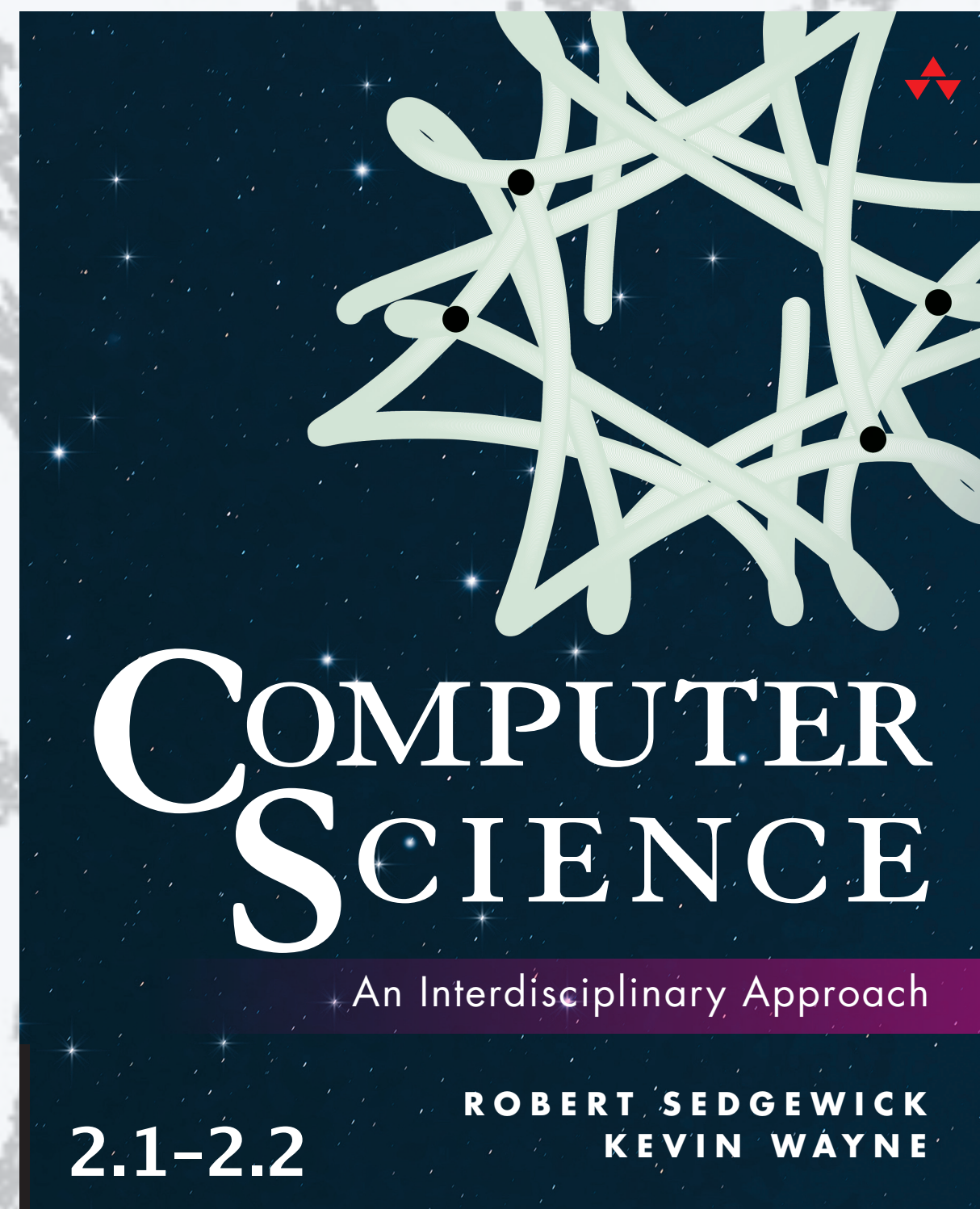


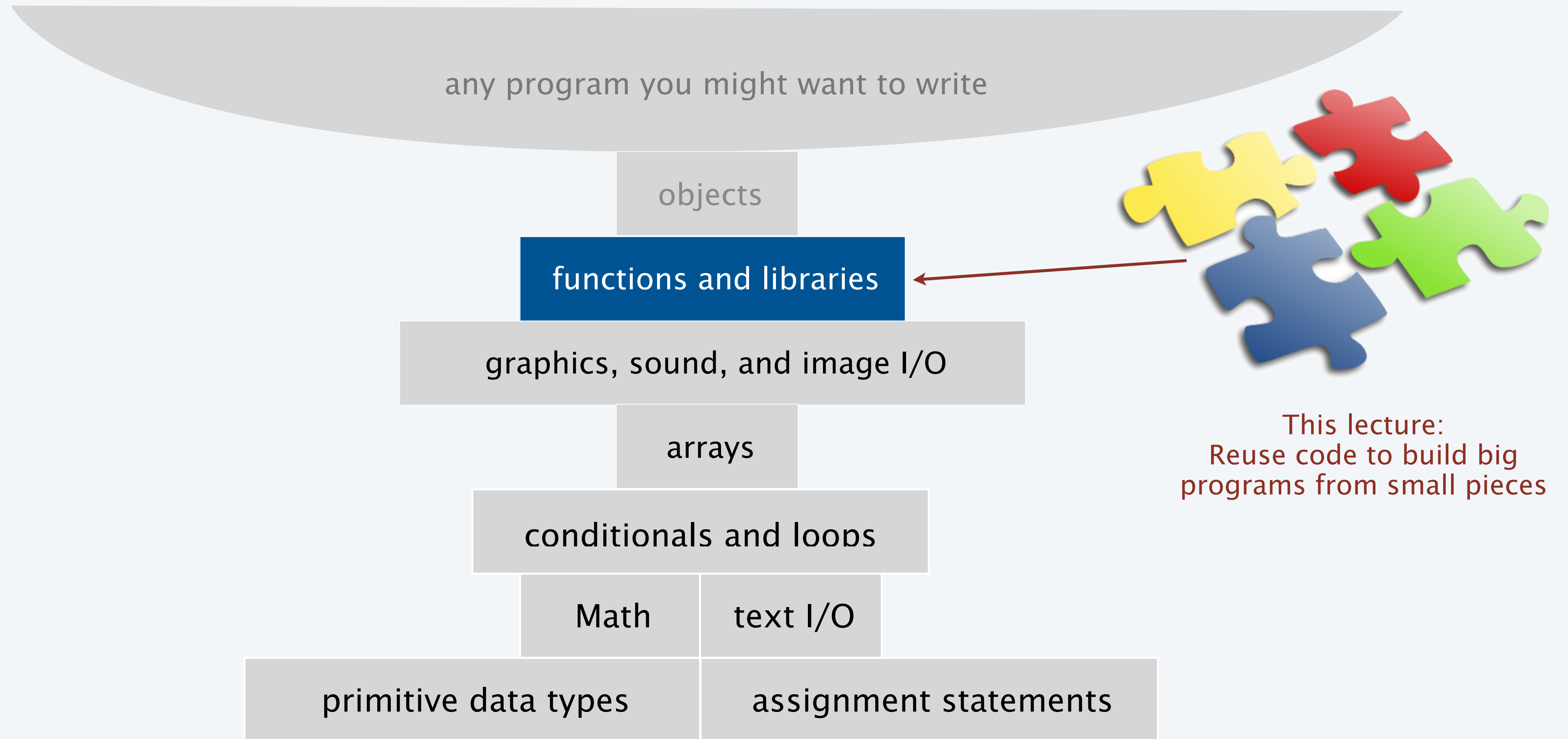


5. Functions and Libraries

5. Functions and Libraries

- **Basic concepts**
- Case study: Digital audio
- Application: Gaussian distribution
- Modular programming and libraries

Context: basic building blocks for programming



Functions, libraries, and modules

Modular programming

- Organize programs as independent **modules** that do a job together.
- Why? Easier to **share and reuse code** to build bigger programs.

Facts of life

- Support of modular programming has been a holy grail for decades.
- Ideas can conflict and get highly technical in the real world.



Def. A **library** is a set of functions.

↑
for purposes of this lecture

Def. A **module** is a .java file.

↑
for purposes of this course

For now. Libraries and modules are the *same thing*: .java files containing sets of functions.

Later. Modules implement *data types* (stay tuned).

Functions (static methods)

Java *function* ("aka *static method*")

- Takes zero or more *input* arguments.
- Returns zero or one *output* value.
- May cause *side effects* (e.g., output to standard draw).

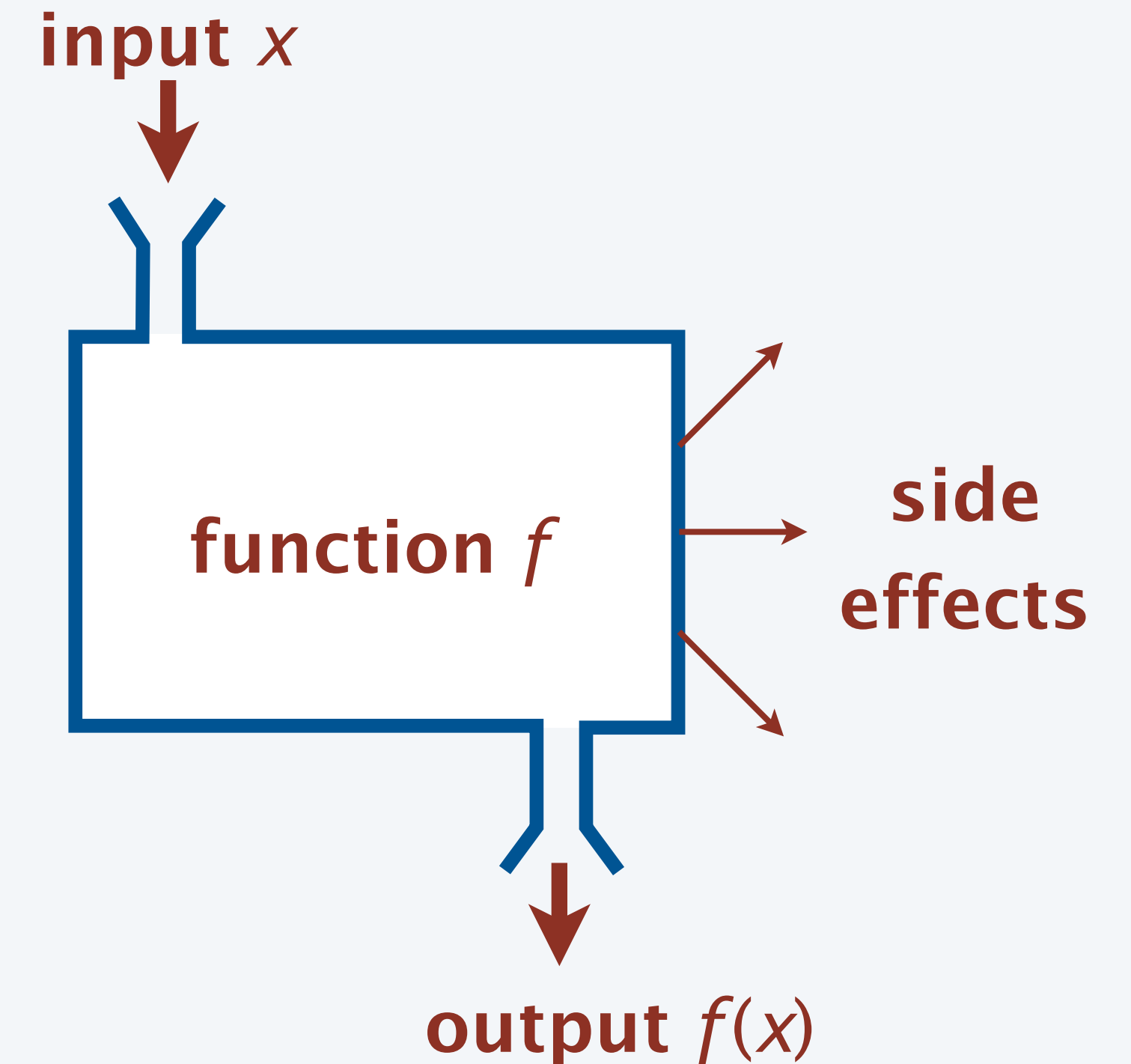
Java functions are *more general* than mathematical functions

Applications

- Scientists use mathematical functions to calculate formulas.
- Programmers use functions to build modular programs.
- *You* use functions for both.

Examples seen so far

- Built-in functions: `Math.random()`, `Math.abs()`, `Integer.parseInt()`.
- Our I/O libraries: `StdIn.readInt()`, `StdDraw.line()`, `StdAudio.play()`.
- User-defined functions: `main()`.



Anatomy of a Java static method

To implement a function (static method)

- Create a *name*.
- Declare type and name of *argument(s)*.
- Specify type for *return value*.
- Implement *body* of method.
- Finish with *return* statement.

The diagram illustrates the components of a Java static method signature and its body. The signature is `public static double sqrt(double c, double eps)`. Annotations point to various parts: 'return type' points to `double`; 'method name' points to `sqrt`; 'argument declarations' points to `double c, double eps`. The body is enclosed in curly braces and contains an `if` statement, variable declarations, a `while` loop, and a `return` statement. A box highlights the entire body, with an arrow labeled 'body of sqrt()' pointing to it. Another box highlights the `return t;` statement, with an arrow labeled 'return statement' pointing to it. The entire signature is labeled 'the method's signature'.

```
public static double sqrt(double c, double eps)
{
    if (c < 0) return Double.NaN;
    double t = c;
    while (Math.abs(t - c/t) > eps * t)
        t = (c/t + t) / 2.0;
    return t;
}
```

return type

method name

argument declarations

the method's *signature*

body of `sqrt()`

return statement

Anatomy of a Java library

A **library** is a set of functions.

Note: We are using our `sqrt()` from Lecture 2 here to illustrate the basics with a familiar function.

Our focus is on control flow here. See Lecture 2 for technical details.

You can use `Math.sqrt()`.

`sqrt()` method

module named
`Newton.java`

`main()` method

```
public class Newton ← library/module name
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }

    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
            StdOut.println(sqrt(a[i], 1e-3));
    }
}
```

Key point. Functions provide a *new way* to control the flow of execution.

Scope

Def. The **scope** of a variable is the code that can refer to it by name.

```
public class Newton
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }

    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
            StdOut.println(sqrt(a[i], 1e-3));
    }
}
```

scope of c and eps →

scope of t →

scope of a →

cannot refer to a or i in this code

cannot refer to c, eps, or t in this code

In a Java library, a variable's scope is the code following its declaration, in the same block.

two *different* variables named i each with scope limited to a single for loop

every loops are a scope

Best practice. Declare variables so as to *limit* their scope.

Flow of control

```
public class Newton
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }

    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
        {
            double x = sqrt(a[i], 1e-3);
            StdOut.println(x);
        }
    }
}
```

need to return a double value

don't need to return value

just the value, not equal, will not be influence

Summary of flow control for a function call

- Control transfers to the function code.
- Argument variables are declared and initialized with the given values.
- Function code is executed.
- Control transfers back to the calling code (with return value assigned in place of the function name in the calling code).

↑
“pass by value”
(other methods used in other systems)

Note. OS calls main() on java command

Function call flow of control trace

```
public class Newton
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }
    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
        {
            double x = sqrt(a[i], 1e-3);
            StdOut.println(x);
        }
    }
}
```

c	t
3.0	3.0
	2.0
	1.75
	1.732

i	a[i]	x
0	1.0	1.000
1	2.0	1.414
2	3.0	1.732
3		

```
% java Newton 1 2 3
1.000
1.414
1.732
```

Pop quiz 1a on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1a
{
    public static int cube(int i)
    {
        int j = i * i * i;
        return j;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

Pop quiz 1a on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1a
{
    public static int cube(int i)
    {
        int j = i * i * i;
        return j;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Takes *N* from the command line, then prints cubes of integers from 1 to *N*

```
% javac PQfunctions1a.java
% java PQfunctions1a 6

1 1
2 8
3 27
4 64
5 125
6 216
```

Pop quiz 1b on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1b
{
    public static int cube(int i)
    {
        int i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

Pop quiz 1b on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1b
{
    public static int cube(int i)
    {
        int i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Won't compile. Argument variable `i` is declared and initialized for function block, so the name cannot be reused.

```
% javac PQfunctions1b.java
```

```
PQfunctions1b.java:5: i is already defined in cube(int)
```

```
    int i = i * i * i;
```

```
        ^
```

```
1 error
```

Pop quiz 1c on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1c
{
    public static int cube(int i)
    {
        i = i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

Pop quiz 1c on functions

Q. What happens when you compile and run the following code?

```
public class PQ6_1c
{
    public static int cube(int i)
    {
        i = i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Won't compile. Need return statement.

```
% javac PQfunctions1c.java
PQfunctions1c.java:6: missing return statement
    }
    ^
1 error
```

Pop quiz 1d on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1d
{
    public static int cube(int i)
    {
        i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

Pop quiz 1d on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1d
{
    public static int cube(int i)
    {
        i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Works. The `i` in `cube()` is

- Declared and initialized as an argument.
- Different from the `i` in `main()`.

BUT changing values of function arguments is sufficiently confusing to be deemed bad style for this course.

```
% javac PQfunctions1d.java
```

```
% java PQfunctions1d 6
```

```
1 1
```

```
2 8
```

```
3 27
```

```
4 64
```

```
5 125
```

```
6 216
```

Pop quiz 1e on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1e
{
    public static int cube(int i)
    {
        return i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

Pop quiz 1e on functions

Q. What happens when you compile and run the following code?

```
public class PQfunctions1e
{
    public static int cube(int i)
    {
        return i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Works fine. Preferred (compact) code.

```
% javac PQfunctions1e.java
% java PQfunctions1e 6

1 1
2 8
3 27
4 64
5 125
6 216
```

COMPUTER SCIENCE

SEDGEWICK / WAYNE

PART I: PROGRAMMING IN JAVA

Image sources

http://upload.wikimedia.org/wikipedia/commons/b/ba/Working_Together_Teamwork_Puzzle_Concept.jpg

<http://pixabay.com/en/ball-puzzle-pieces-of-the-puzzle-72374/>

http://upload.wikimedia.org/wikipedia/commons/e/ef/Ben_Jigsaw_Puzzle_Puzzle_Puzzle.png

[http://en.wikipedia.org/wiki/Function_\(mathematics\)#mediaviewer/File:Function_machine2.svg](http://en.wikipedia.org/wiki/Function_(mathematics)#mediaviewer/File:Function_machine2.svg)

5. Functions and Libraries

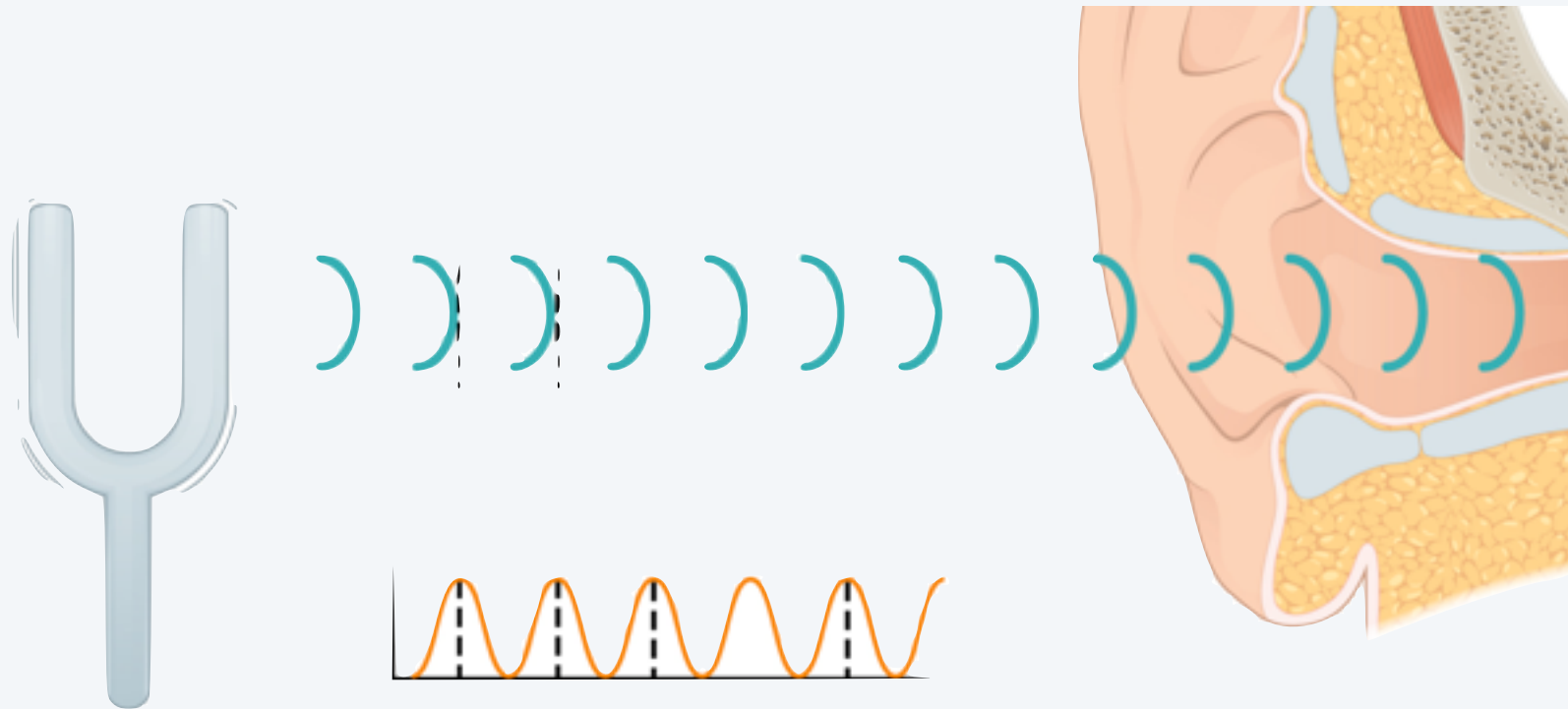
- Basic concepts
- **Case study: Digital audio**
- Application: Gaussian distribution
- Modular programming

Crash course in sound

Sound is the perception of the vibration of molecules.

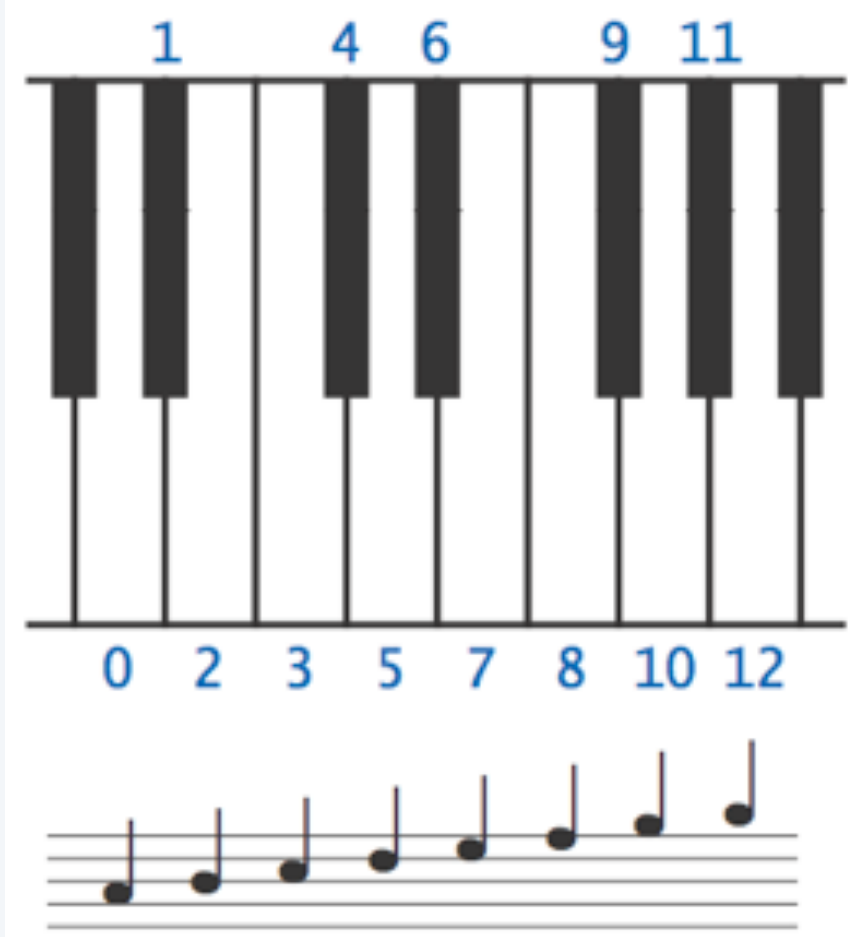
A musical tone is a steady periodic sound.

A pure tone is a sinusoidal waveform.



Western musical scale

- Concert A is 440 Hz.
- 12 notes, logarithmic scale.



<i>pitch</i>	<i>i</i>	<i>frequency</i> ($440 \cdot 2^{i/12}$)	<i>sinusoidal waveform</i>
A	0	440	
A# / Bb	1	466.16	
B	2	493.88	
C	3	523.25	
C# / Db	4	554.37	
D	5	587.33	
D# / Eb	6	622.25	
E	7	659.26	
F	8	698.46	
F# / Gb	9	739.99	
G	10	783.99	
G# / Ab	11	830.61	
A	12	880	

Digital audio

To represent a wave, *sample* at regular intervals and save the values in an array.

← same as when plotting
a function
(previous lecture)

1/40 second of concert A

CD standard
Used in StdAudio

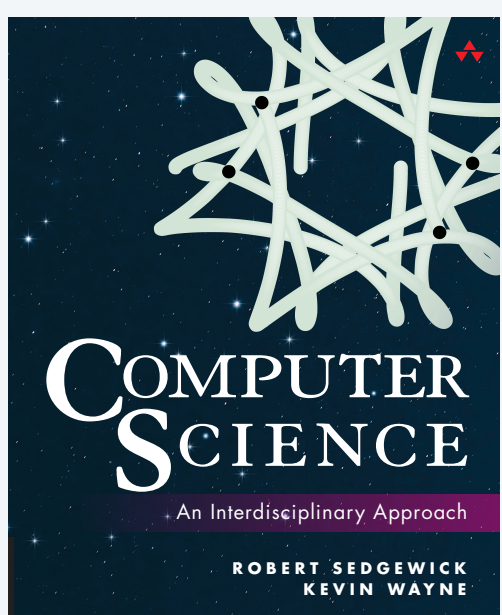
<i>samples/sec</i>	<i>samples</i>	<i>sampld waveform</i>
5,512	137	
11,025	275	
22,050	551	
44,100	1102	

Bottom line. You can *write programs* to manipulate sound (arrays of double values).

StdAudio library

Developed for this course, also broadly useful

- Play a sound wave (array of double values) on your computer's audio output.
- Convert to and from standard .wav file format.



API

<code>public class StdAudio</code>	
<code>void play(String file)</code>	<i>play the given .wav file</i>
<code>void play(double[] a)</code>	<i>play the given sound wave</i>
<code>void play(double x)</code>	<i>play the sample for 1/44100 second</i>
<code>void save(String file, double[] a)</code>	<i>save to a .wav file</i>
<code>double[] read(String file)</code>	<i>read from a .wav file</i>

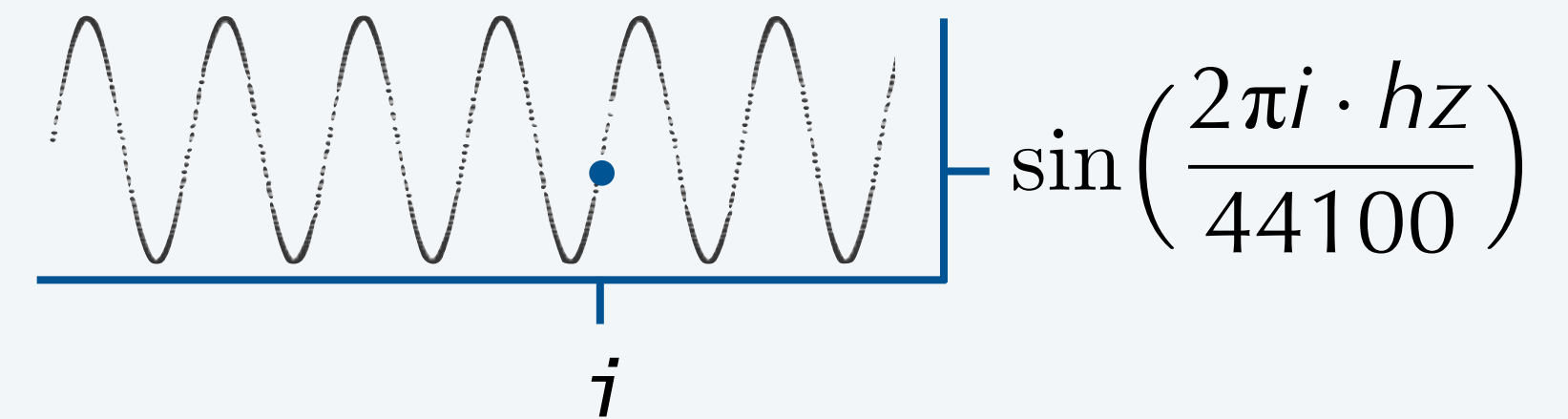
Enables you to *hear the results* of your programs that manipulate sound.

“Hello, World” for StdAudio

这里的中括号的目的是要求方法返回一个
double数组，而不是一个具体的值

```
public class PlayThatNote
{
    public static double[] tone(double hz, double duration)
    {
        int N = (int) (44100 * duration);
        double[] a = new double[N+1];
        for (int i = 0; i <= N; i++)
            a[i] = Math.sin(2 * Math.PI * i * hz / 44100);
        return a;
    }

    public static void main(String[] args)
    {
        double hz = Double.parseDouble(args[0]);
        double duration = Double.parseDouble(args[1]);
        double[] a = tone(hz, duration);
        StdAudio.play(a);
    }
}
```



```
% java PlayThatNote 440.0 3.0
```

```
% java PlayThatNote 880.0 3.0
```

```
% java PlayThatNote 220.0 3.0
```

```
% java PlayThatNote 494.0 3.0
```

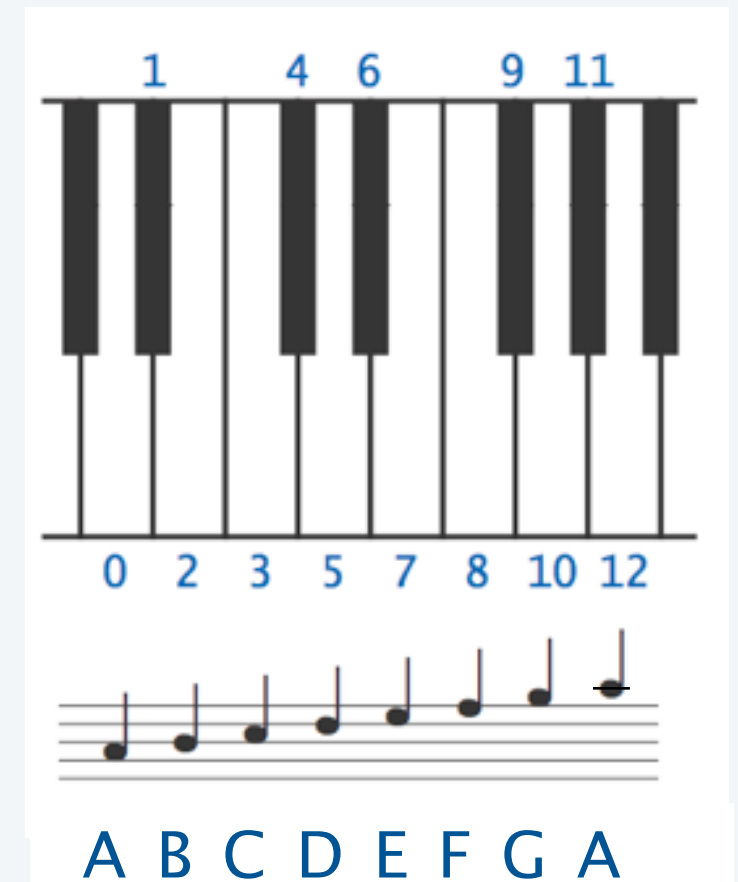
Play that tune

Read a list of tones and durations from standard input to **play a tune**.

```
public class PlayThatTune
{
    public static void main(String[] args)
    {
        double tempo = Double.parseDouble(args[0]);
        while (!StdIn.isEmpty())
        {
            int pitch = StdIn.readInt();
            double duration = StdIn.readDouble() * tempo;
            double hz = 440 * Math.pow(2, pitch / 12.0);
            double[] a = PlayThatNote.tone(hz, duration);
            StdAudio.play(a);
        }
        StdAudio.drain();
    }
}
```

control tempo from command line

```
% more < elise.txt
7 .125
6 .125
7 .125
6 .125
7 .125
2 .125
5 .125
3 .125
0 .25
...
```



```
% java PlayThatTune 2.0 < elise.txt
```



```
% java PlayThatTune 1.0 < elise.txt
```

Pop quiz 2 on functions

Q. What sound does the following program produce?

```
public class PQfunctions2
{
    public static void main(String[] args)
    {
        int N = (int) (44100 * 11);
        double[] a = new double[N+1];
        for (int i = 0; i <= N; i++)
            a[i] = Math.random();
        StdAudio.play(a);
    }
}
```

Pop quiz 2 on functions

Q. What sound does the following program produce?

```
public class PQfunctions2
{
    public static void main(String[] args)
    {
        int N = (int) (44100 * 11.0);
        double[] a = new double[N+1];
        for (int i = 0; i <= N; i++)
            a[i] = Math.random();
        StdAudio.play(a);
    }
}
```

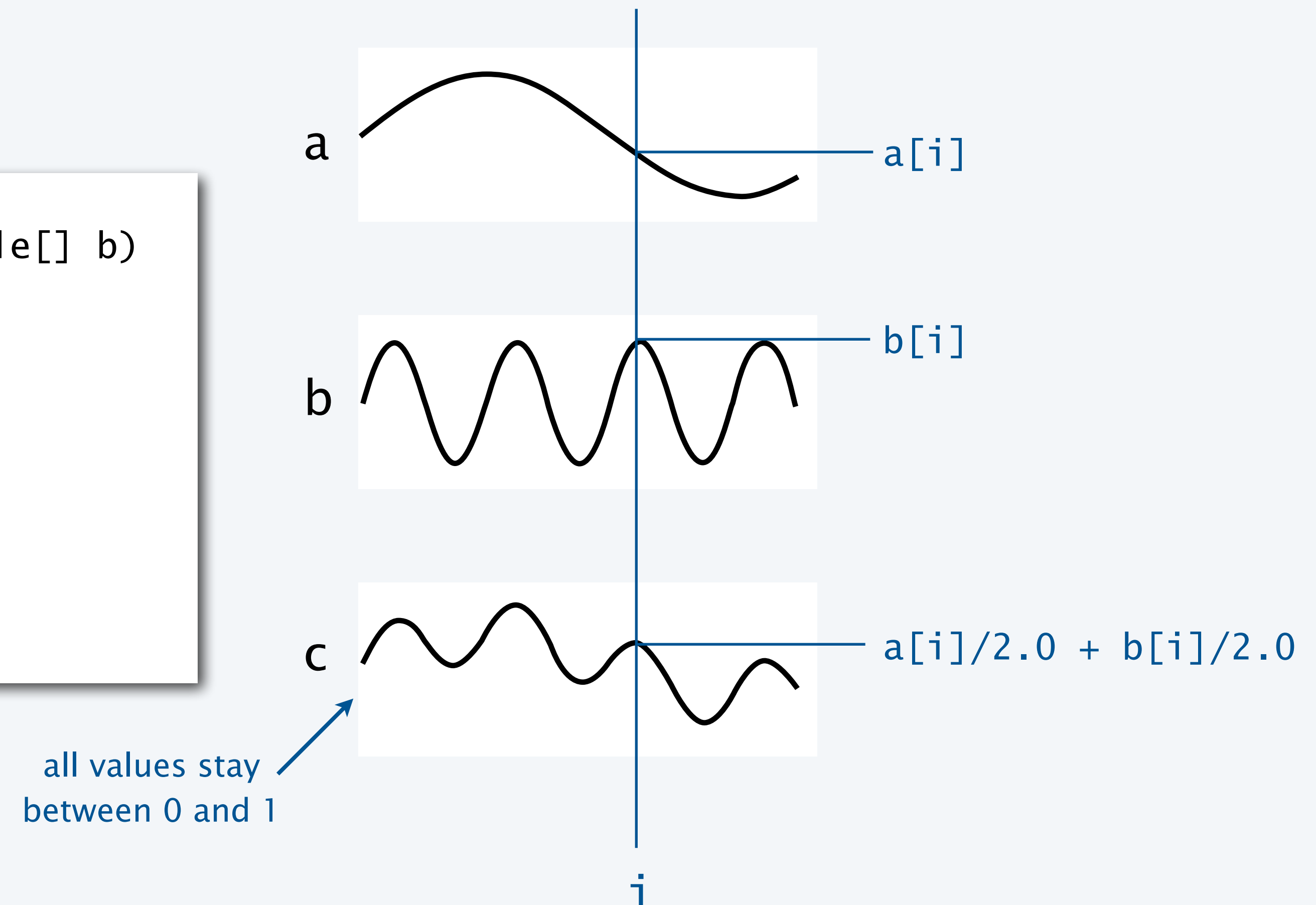
A. 11 seconds of pure *noise*.

```
% java PQfunctions2.java
```

Play that chord

Produce chords by *averaging* waveforms.

```
public static double[] avg(double[] a, double[] b)
{
    double[] c = new double[a.length];
    for (int i = 0; i < a.length; i++)
        c[i] = a[i]/2.0 + b[i]/2.0;
    return c;
}
```

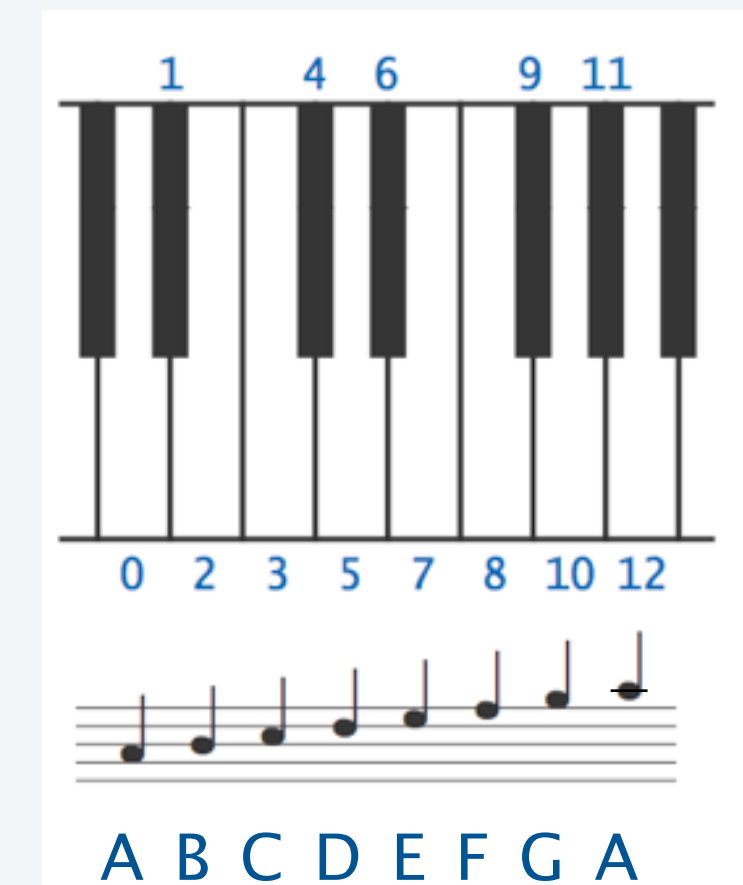


Play that chord implementation

```
public class PlayThatChord
{
    public static double[] avg(double[] a, double[] b)
    { /* See previous slide. */ }

    public static double[] chord(int pitch1, int pitch2, double d)
    {
        double hz1 = 440.0 * Math.pow(2, pitch1 / 12.0);
        double hz2 = 440.0 * Math.pow(2, pitch2 / 12.0);
        double[] a = PlayThatNote.tone(hz1, d);
        double[] b = PlayThatNote.tone(hz2, d);
        return avg(a, b);
    }

    public static void main(String[] args)
    {
        int pitch1 = Integer.parseInt(args[0]);
        int pitch2 = Integer.parseInt(args[1]);
        double duration = Double.parseDouble(args[2]);
        double[] a = chord(pitch1, pitch2, duration);
        StdAudio.play(a);
    }
}
```



```
% java PlayThatChord 0 3 5.0
```

```
% java PlayThatChord 0 12 5.0
```

Play that tune (deluxe version)

Add harmonics to PlayThatTune to produce a more realistic sound.

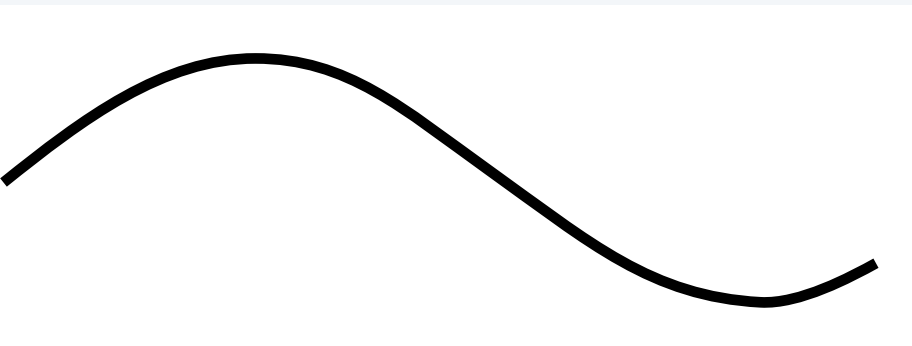
Function to add harmonics to a tone

```
public static double[] note(int pitch, double duration)
{
    double hz = 440.0 * Math.pow(2, pitch / 12.0);
    double[] a = tone(1.0 * hz, duration);
    double[] hi = tone(2.0 * hz, duration);
    double[] lo = tone(0.5 * hz, duration);
    double[] harmonic = avg(hi, lo);
    return avg(a, harmonic);
}
```

```
% java PlayThatTune 1.5 < elise.txt
```

Program 2.1.4 in text (with tempo added)

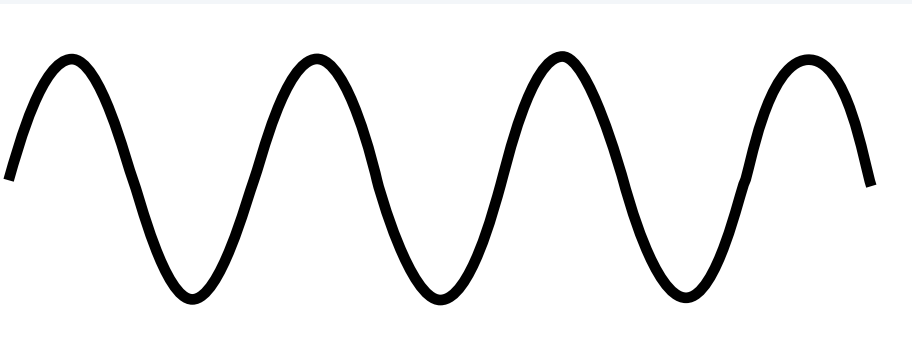
half
frequency



lo



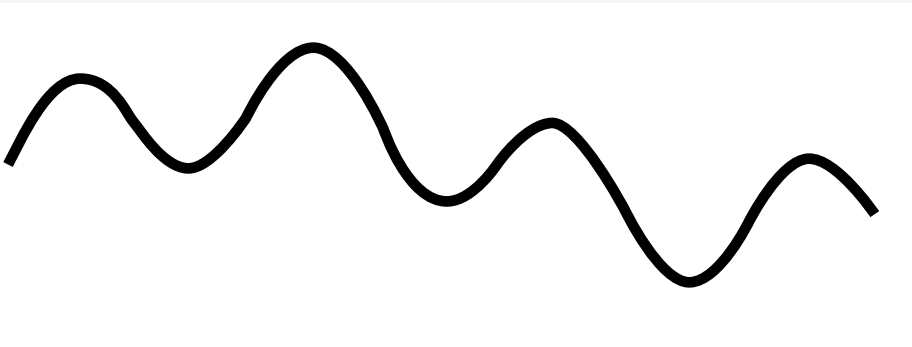
double
frequency



hi

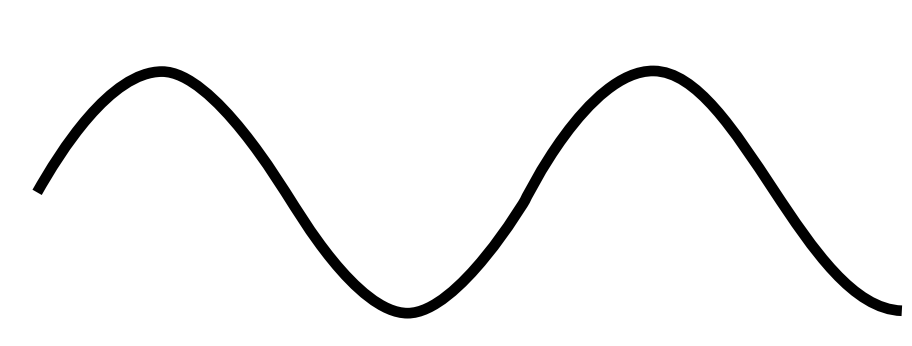


both
harmonics



harmonic

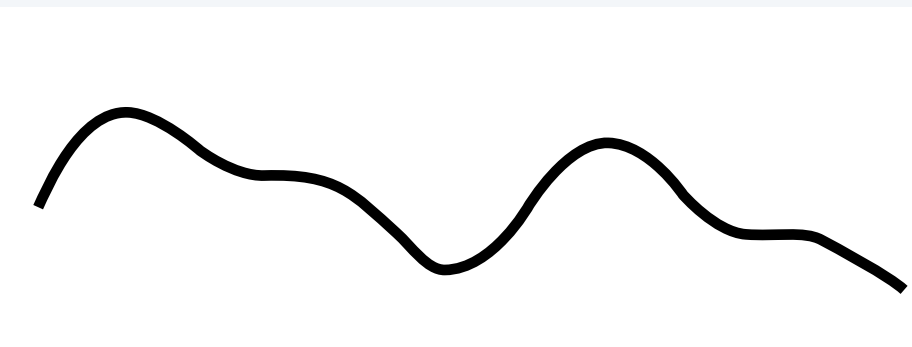
pure
pitch



a



result



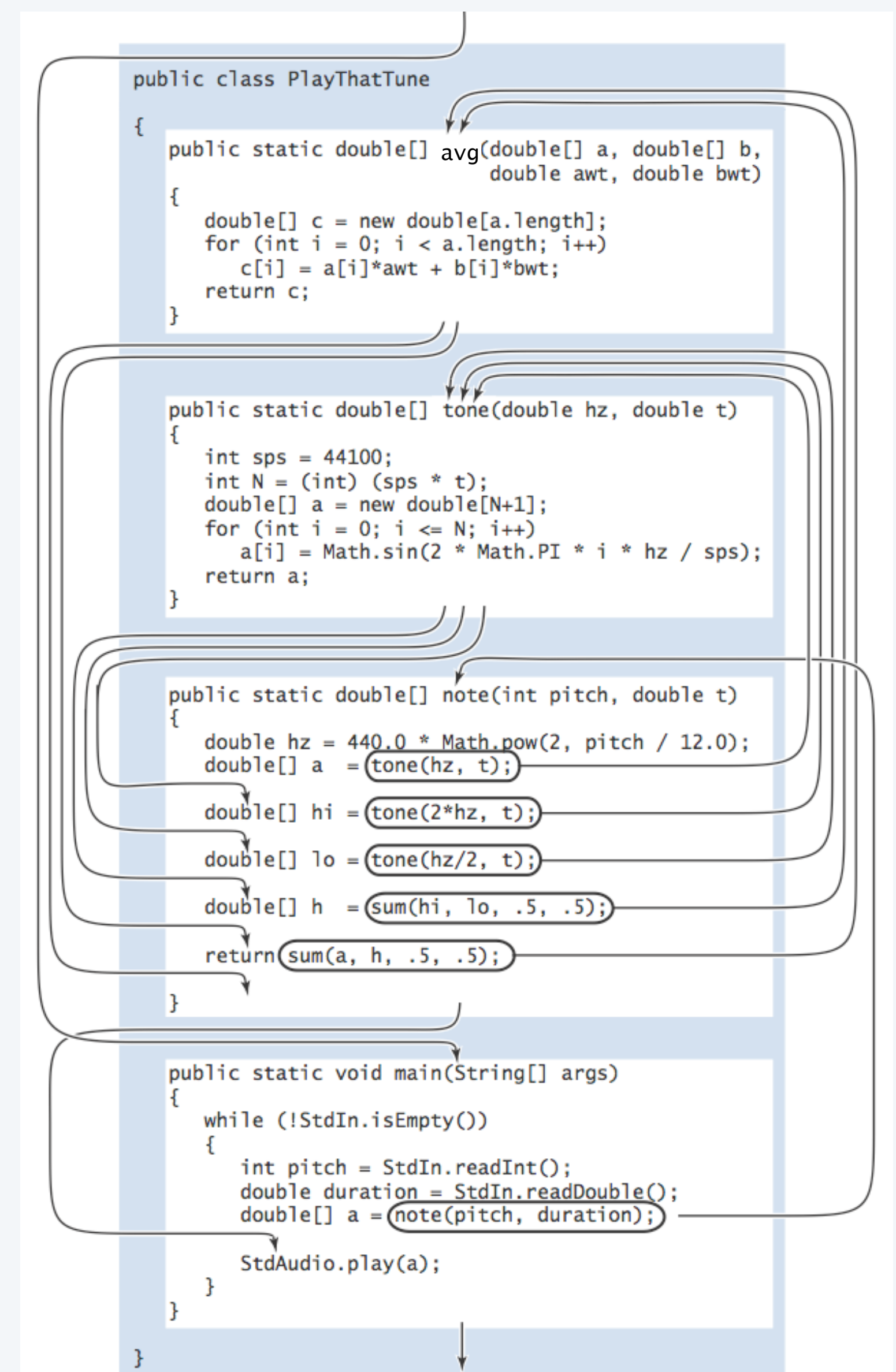
avg(a, harmonic)

Digital audio (summary)



```
% java PlayThatTune 1.5 < entertainer.txt
```

Bottom line. You can write programs to manipulate sound.
This lecture: Case study of the utility of functions.



COMPUTER SCIENCE

SEDGEWICK / WAYNE

PART I: PROGRAMMING IN JAVA

Image sources

http://commons.wikimedia.org/wiki/File:1405_Sound_Waves_and_the_Ear.jpg

[http://en.wikipedia.org/wiki/The_Entertainer_\(rag\)#mediaviewer/File:EntertainerJoplinCover.JPEG](http://en.wikipedia.org/wiki/The_Entertainer_(rag)#mediaviewer/File:EntertainerJoplinCover.JPEG)

<http://cantorion.org/music/497/The-Entertainer-Original-version>

5. Functions and Libraries

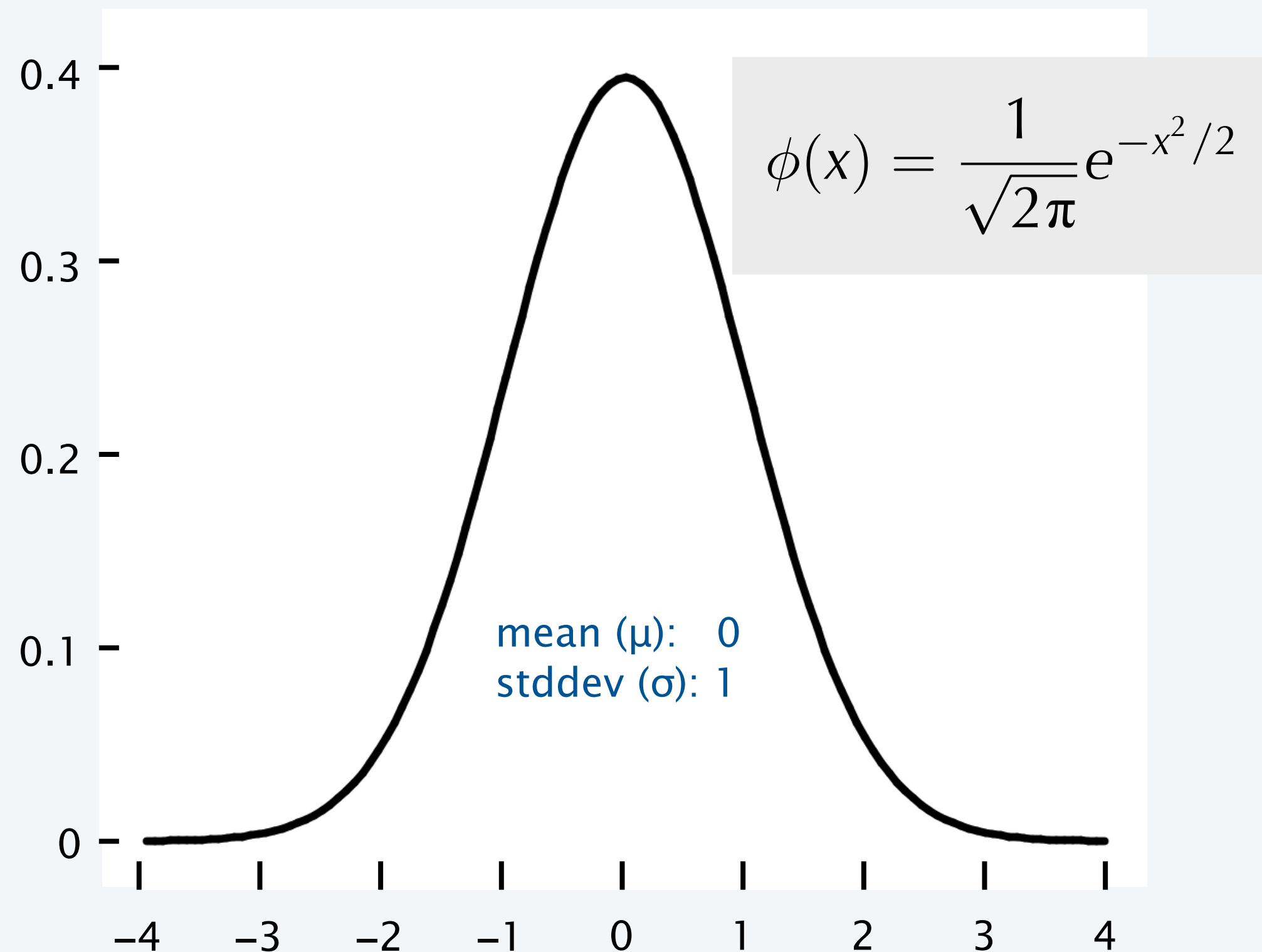
- Basic concepts
- Case study: Digital audio
- **Application: Gaussian distribution**
- Modular programming

Gaussian distribution

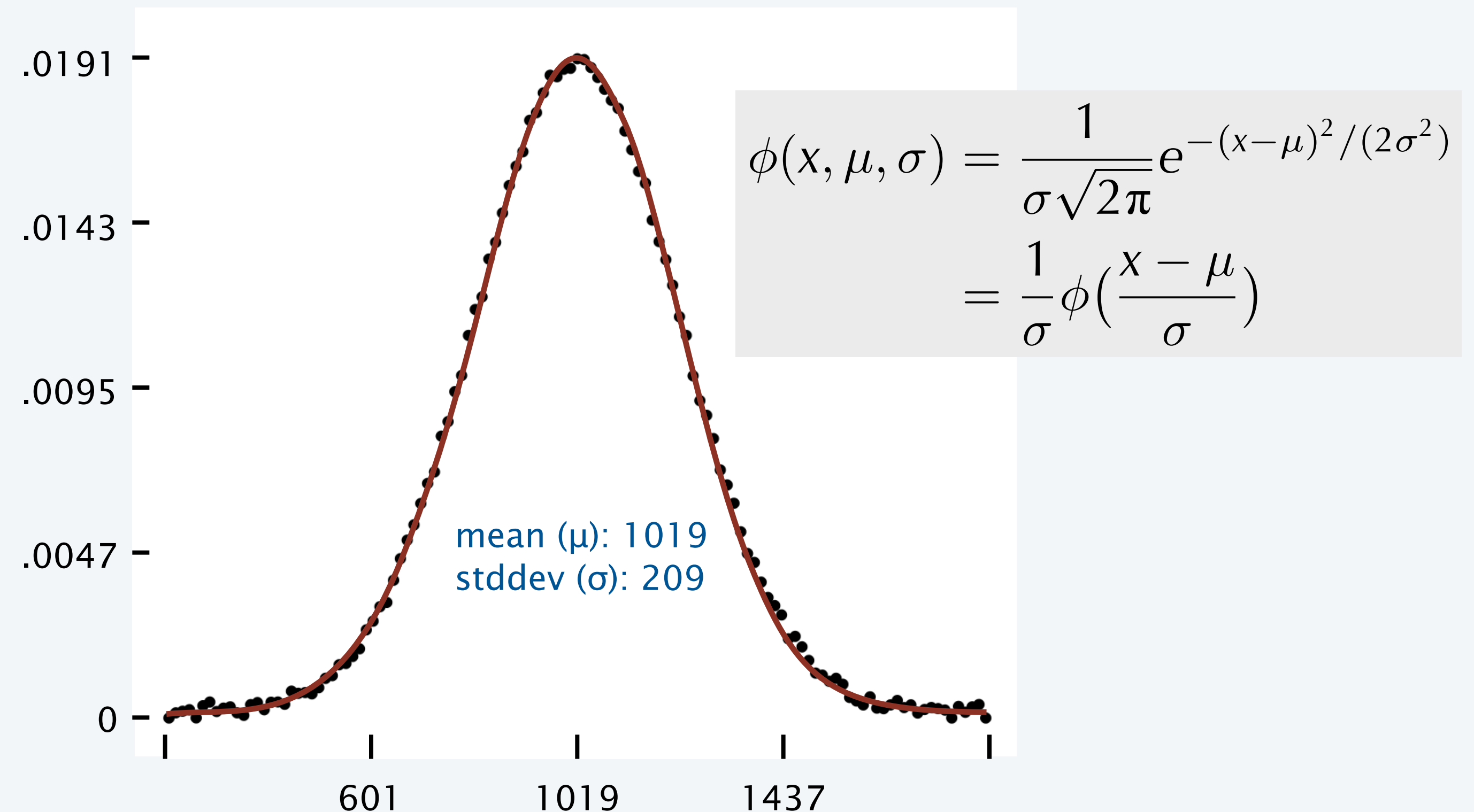
Gaussian distribution.

- A mathematical model used successfully for centuries.
- "Bell curve" fits experimental observations in many contexts.

Gaussian probability density function (pdf)

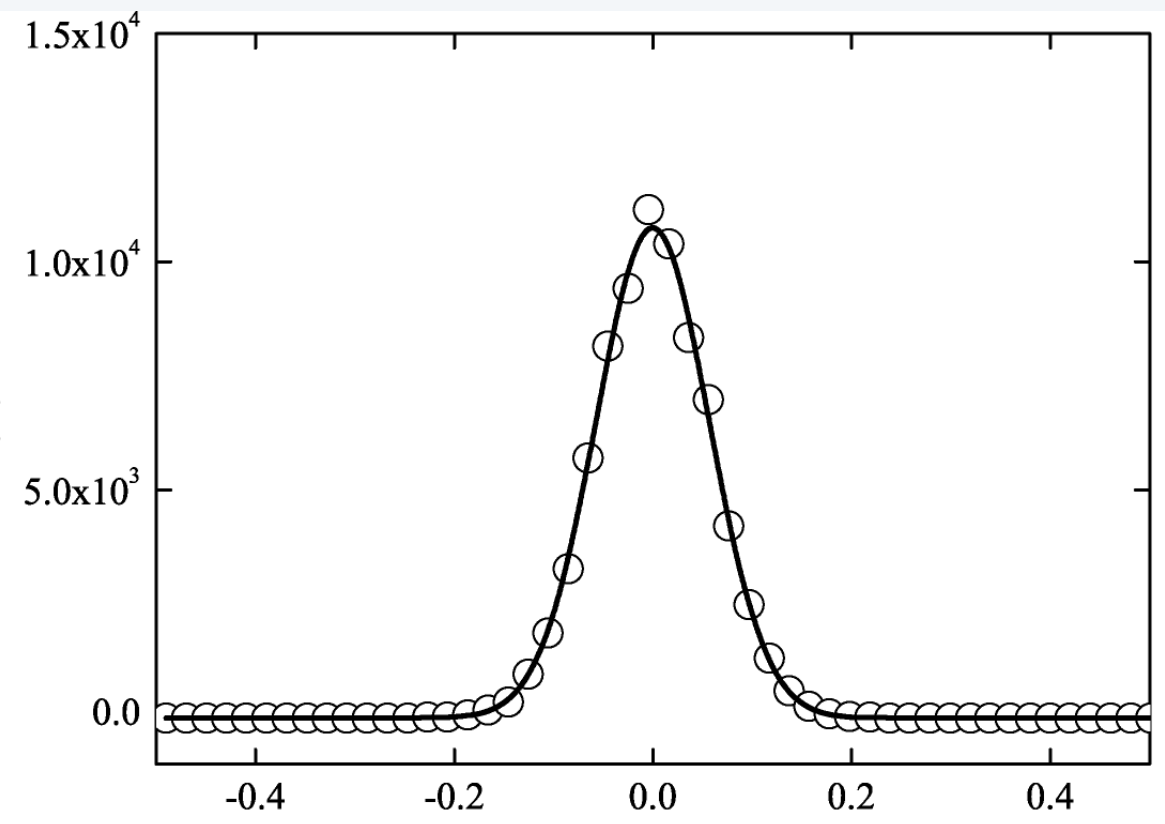


Example: SAT scores in 20xx (verbal + math)



Gaussian distribution in the wild

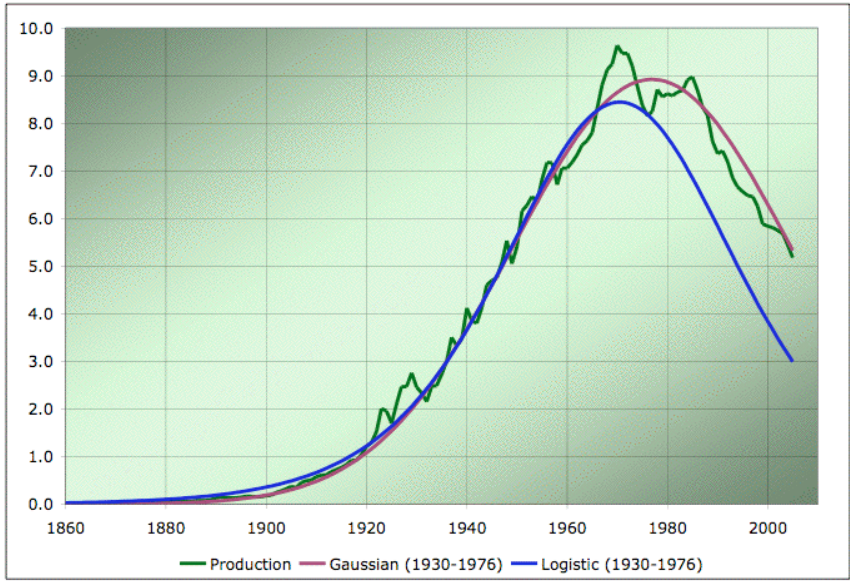
Polystyrene particles in glycerol



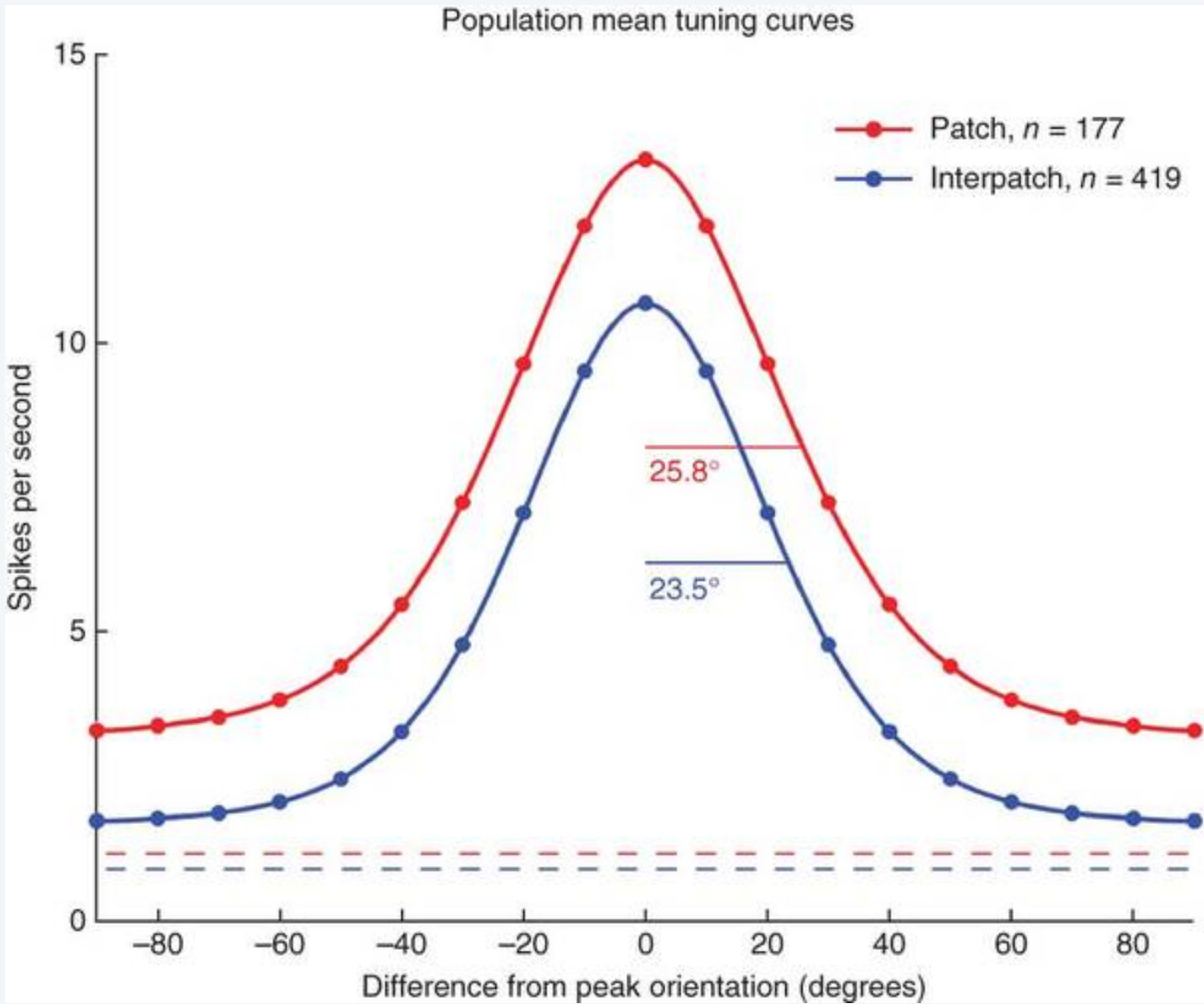
German money



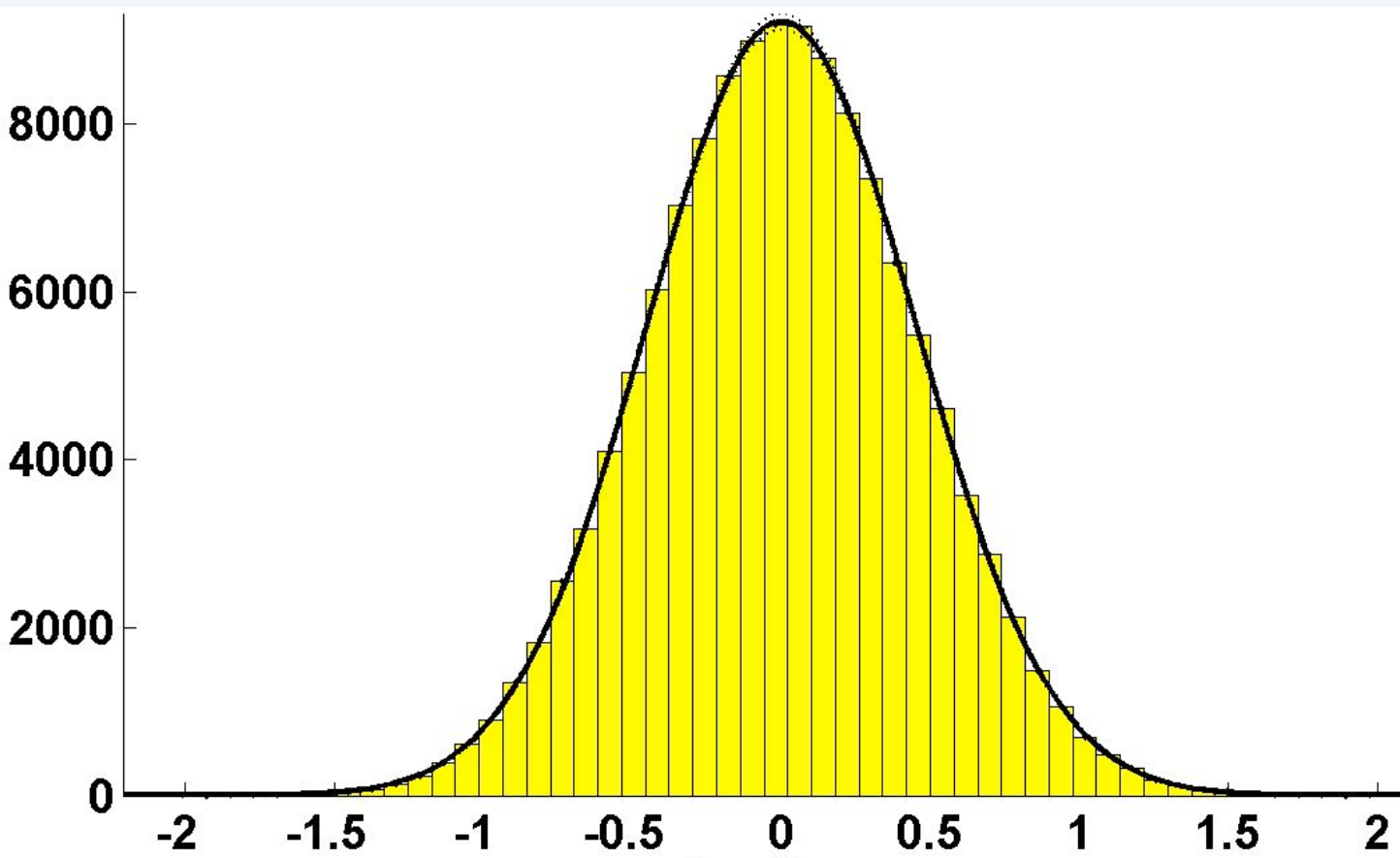
Predicted US oil production



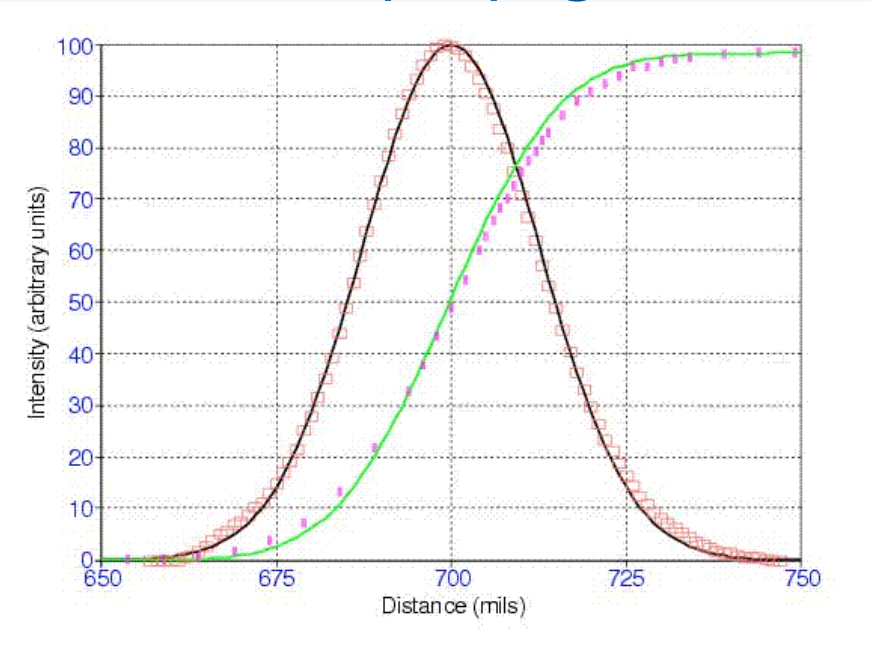
Cytochrome oxidase patches in macaque primary visual cortex



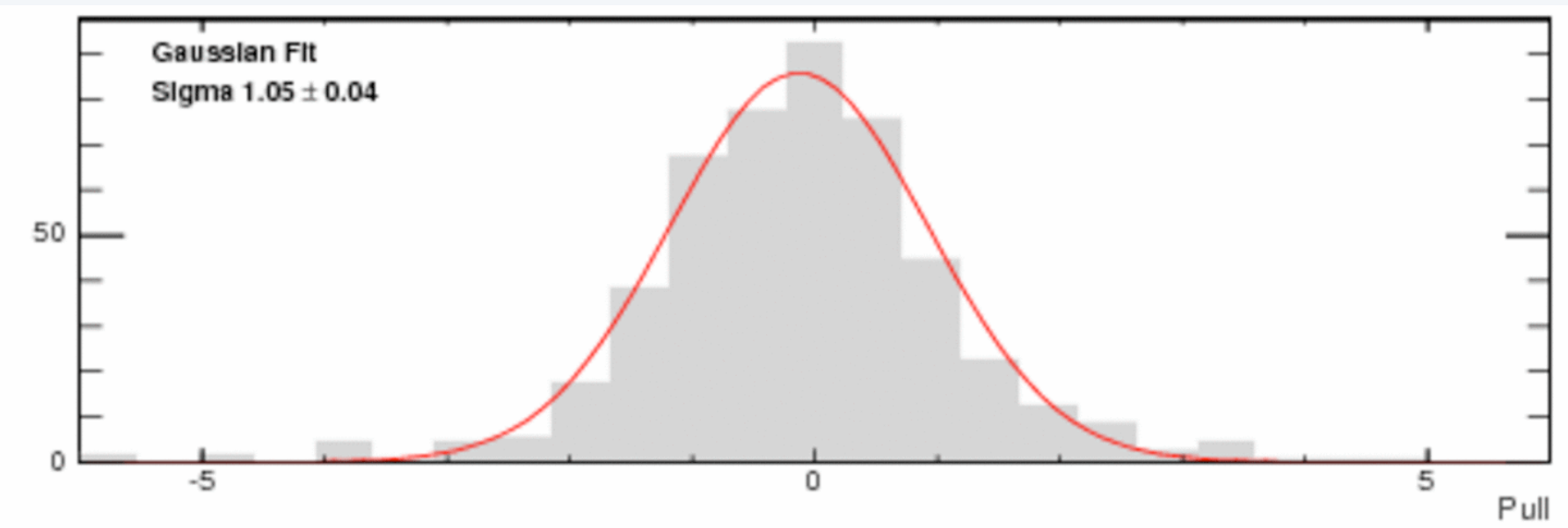
Calibration of optical tweezers



Laser beam propagation



Polarized W bosons from top-quark decay



Defining a library of functions

Q. Is the Gaussian pdf ϕ implemented in Java's Math library?

A. No.

Q. Why not?

A. Maybe because it is so easy for you to do it yourself.

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

$$\phi(x, \mu, \sigma) = \frac{1}{\sigma} \phi\left(\frac{x - \mu}{\sigma}\right)$$

```
public class Gaussian
{
```

```
    public static double pdf(double x)
    {
        return Math.exp(-x*x / 2) / Math.sqrt(2 * Math.PI);
    }
```

```
    public static double pdf(double x, double mu, double sigma)
    {
        return pdf((x - mu) / sigma) / sigma;
    }
```

```
    // Stay tuned for more functions.
```

```
}
```

同名但是有不同的量，属于不同的
function

call a function in another module

module named
Gaussian.java

functions with different signatures
are different (even if names match)

Functions and libraries provide an easy way for any user to *extend* the Java system.

Gaussian cumulative distribution function

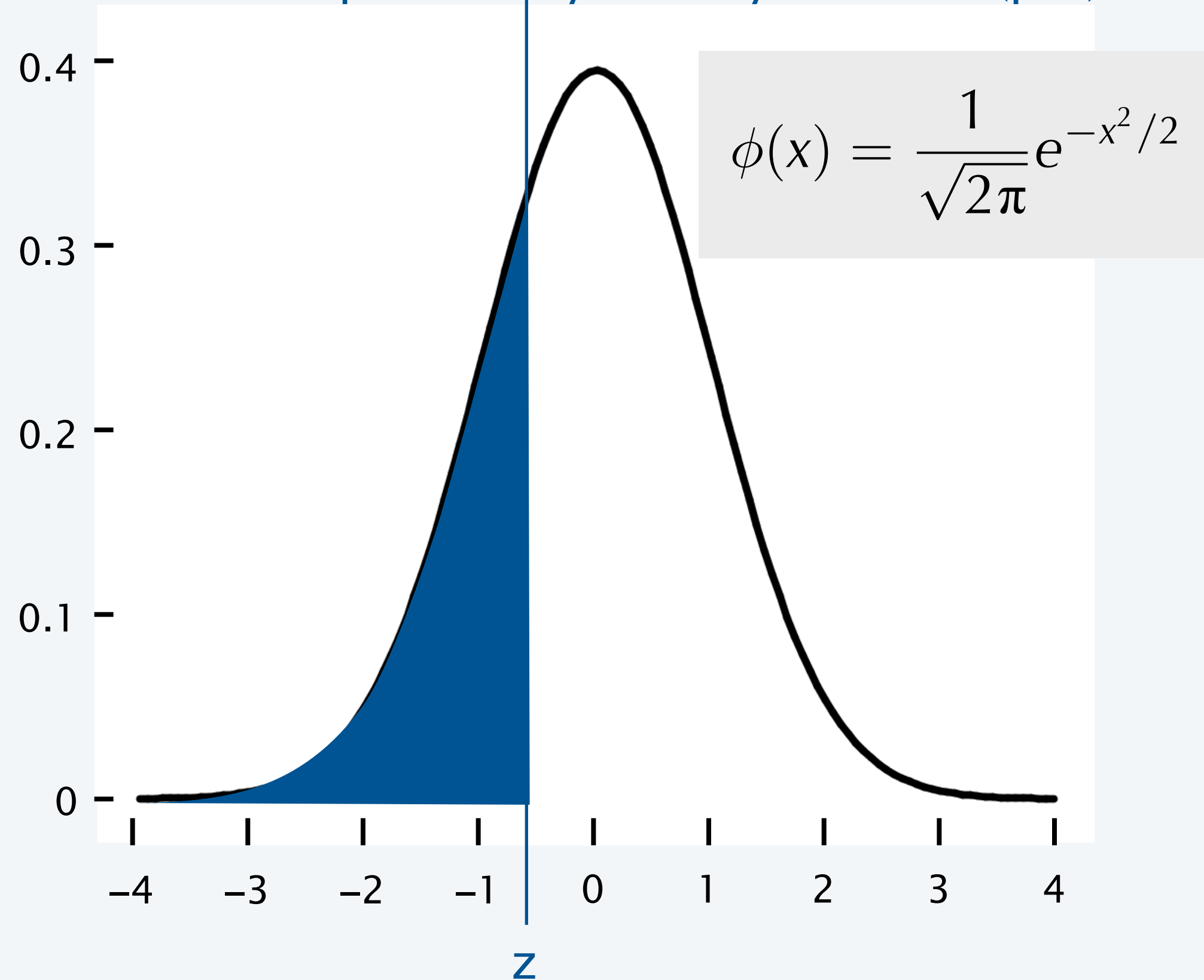
Q. What percentage of the total is less than or equal to z ?

Q. (equivalent). What is the area under the curve to the left of z ?

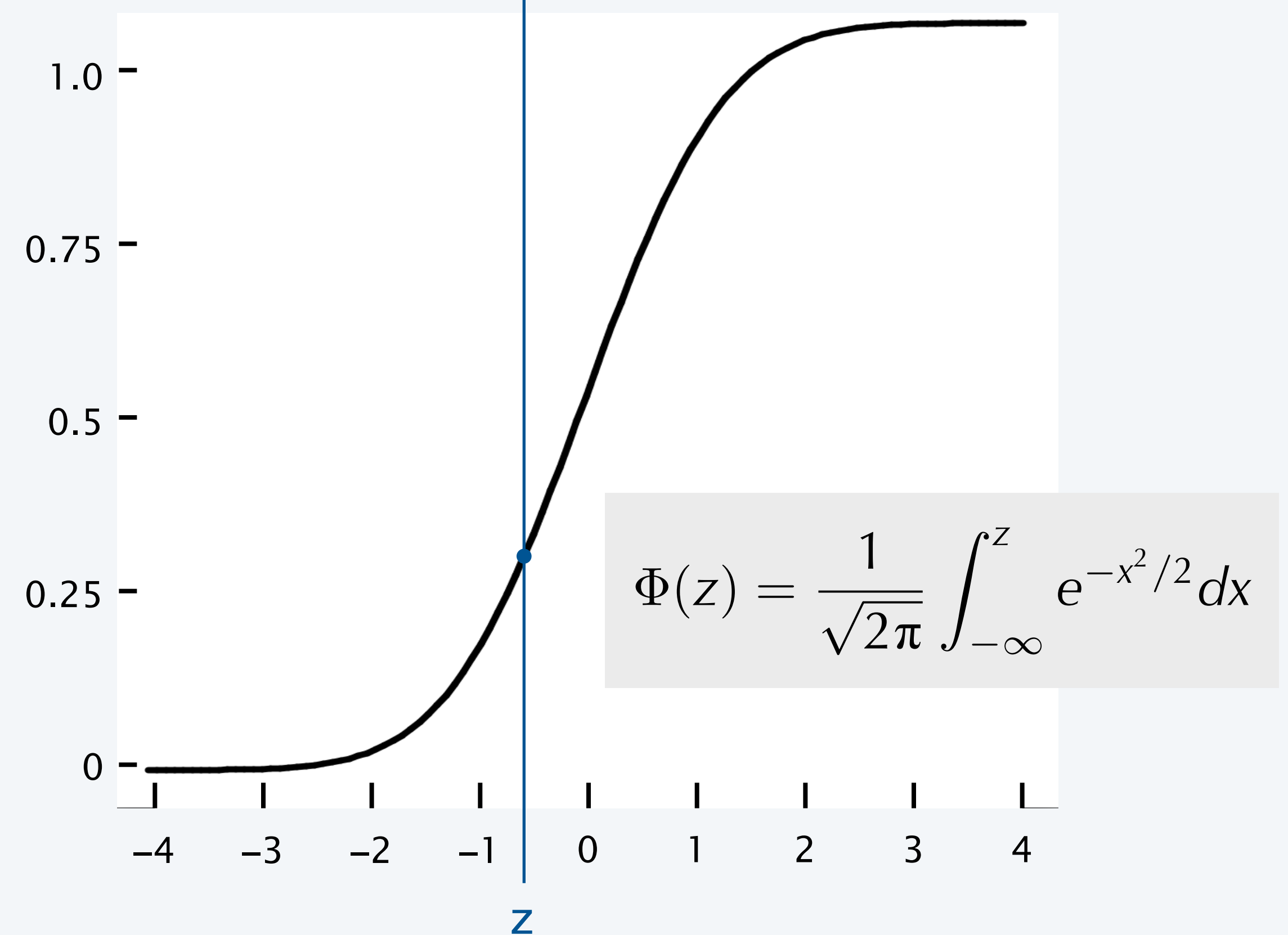
A. Gaussian *cumulative distribution function*.

$$\Phi(x, \mu, \sigma) = \Phi\left(\frac{x - \mu}{\sigma}\right)$$

Gaussian probability density function (pdf)



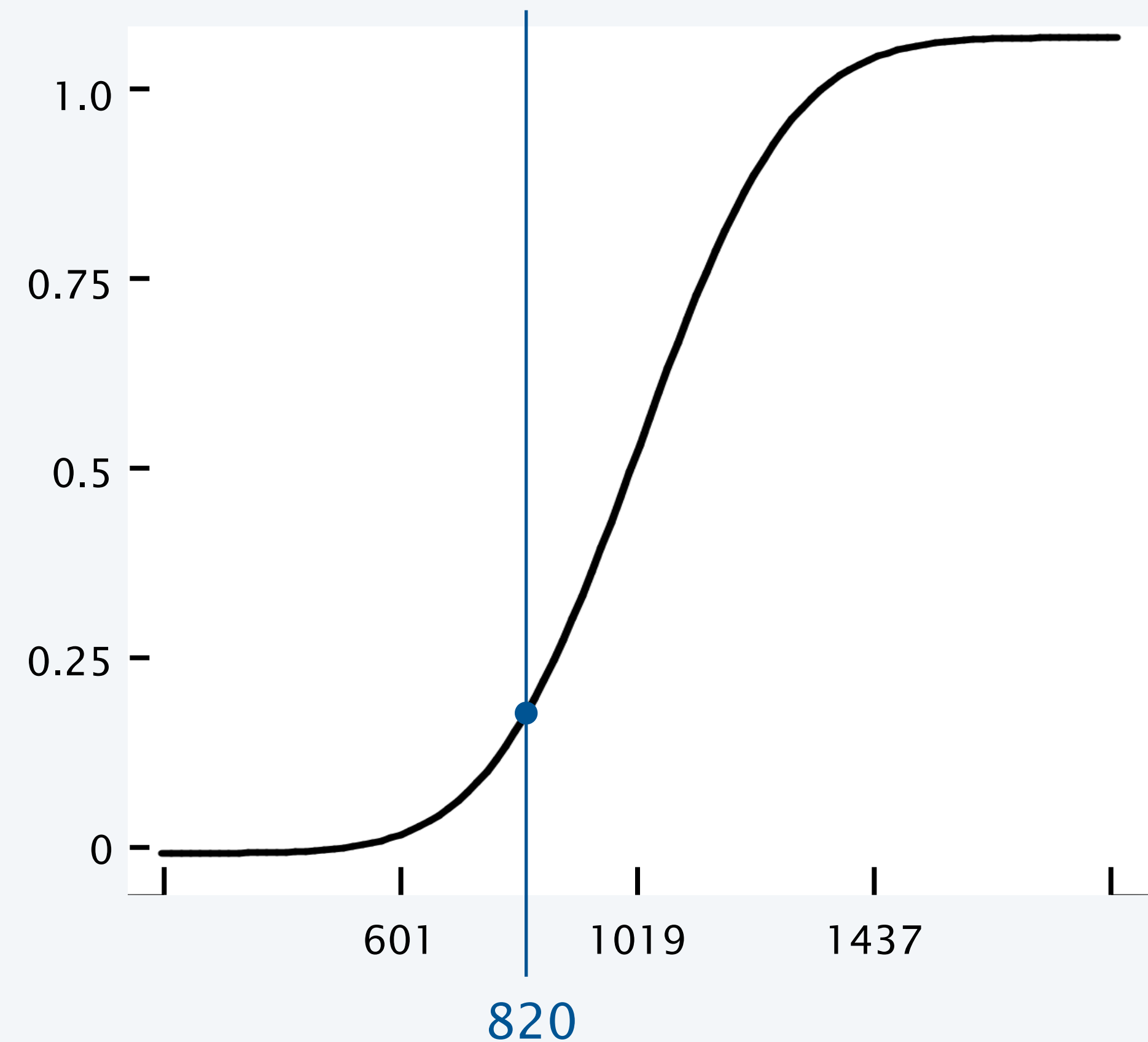
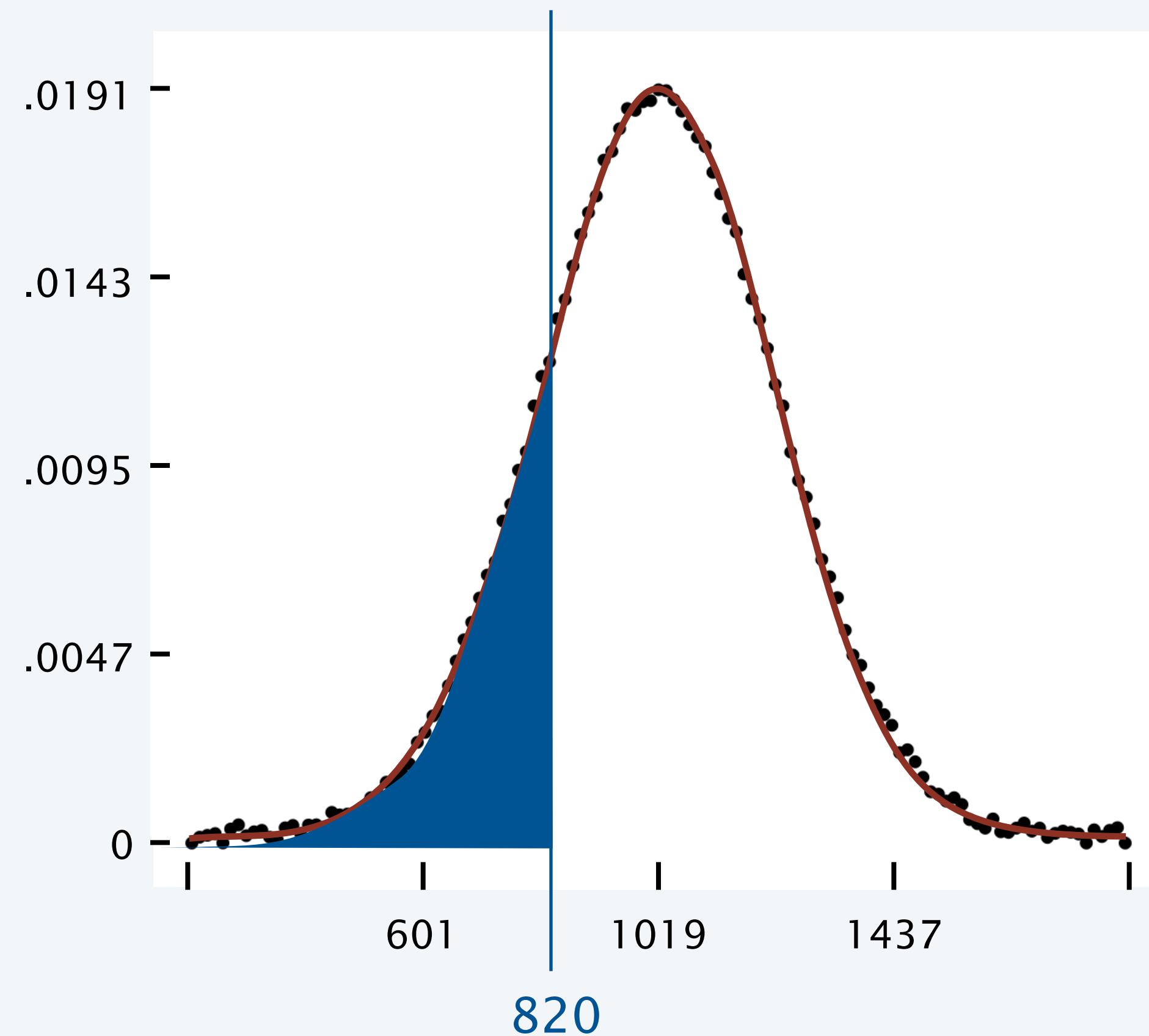
Gaussian cumulative distribution function (cdf)



Typical application: SAT scores

Q. In 20xx NCAA required at least 820 for Division I athletes.
What fraction of test takers did not qualify??

A. About 17%, since $\Phi(820, 1019, 209) = 0.17050966869132111\dots$



Gaussian CDF implementation

Q. No closed form for Gaussian CDF Φ . How to implement?

A. Use **Taylor series**. $\Phi(z) \equiv \int_{-\infty}^z \phi(x) dx = \frac{1}{2} + \phi(z) \left(z + \frac{z^3}{3} + \frac{z^5}{3 \cdot 5} + \frac{z^7}{3 \cdot 5 \cdot 7} + \dots \right)$

```
public static double cdf(double z)
{
    if (z < -8.0) return 0.0;
    if (z > 8.0) return 1.0;
    double sum = 0.0, term = z;
    for (int i = 3; sum + term != sum; i += 2)
    {
        sum = sum + term;
        term = term * z * z / i;
    }
    return 0.5 + sum * pdf(z);    ← accurate to 15 places
}

public static double cdf(double z, double mu, double sigma)
{ return cdf((z - mu) / sigma); }
```

Bottom line. 1,000 years of mathematical formulas at your fingertips.

Summary: a library for Gaussian distribution functions

Best practice

- Test all code at least once in main().
- Also have it do something useful.

Q. What fraction of SAT test takers did not qualify for NCAA participation in 20xx?

```
% java Gaussian 820 1019 209
0.17050966869132111
```

Fun fact

We can use cdf() to evaluate randomness in submitted programs.

Bottom line

YOU can build a layer of abstraction to use in any future program.

```
public class Gaussian
{
    public static double pdf(double x)
    { return Math.exp(-x*x / 2) / Math.sqrt(2 * Math.PI); }
    public static double pdf(double x, double mu, double sigma)
    { return pdf((x - mu) / sigma) / sigma; }
    public static double cdf(double z)
    {
        if (z < -8.0) return 0.0;
        if (z > 8.0) return 1.0;
        double sum = 0.0, term = z;
        for (int i = 3; sum + term != sum; i += 2)
        {
            sum = sum + term;
            term = term * z * z / i;
        }
        return 0.5 + sum * pdf(z);
    }
    public static double cdf(double z, double mu, double sigma)
    { return cdf((z - mu) / sigma); }

    public static void main(String[] args)
    {
        double z = Double.parseDouble(args[0]);
        double mu = Double.parseDouble(args[1]);
        double sigma = Double.parseDouble(args[2]);
        StdOut.println(cdf(z, mu, sigma));
    }
}
```

标准化之后的正态分布

Using a library

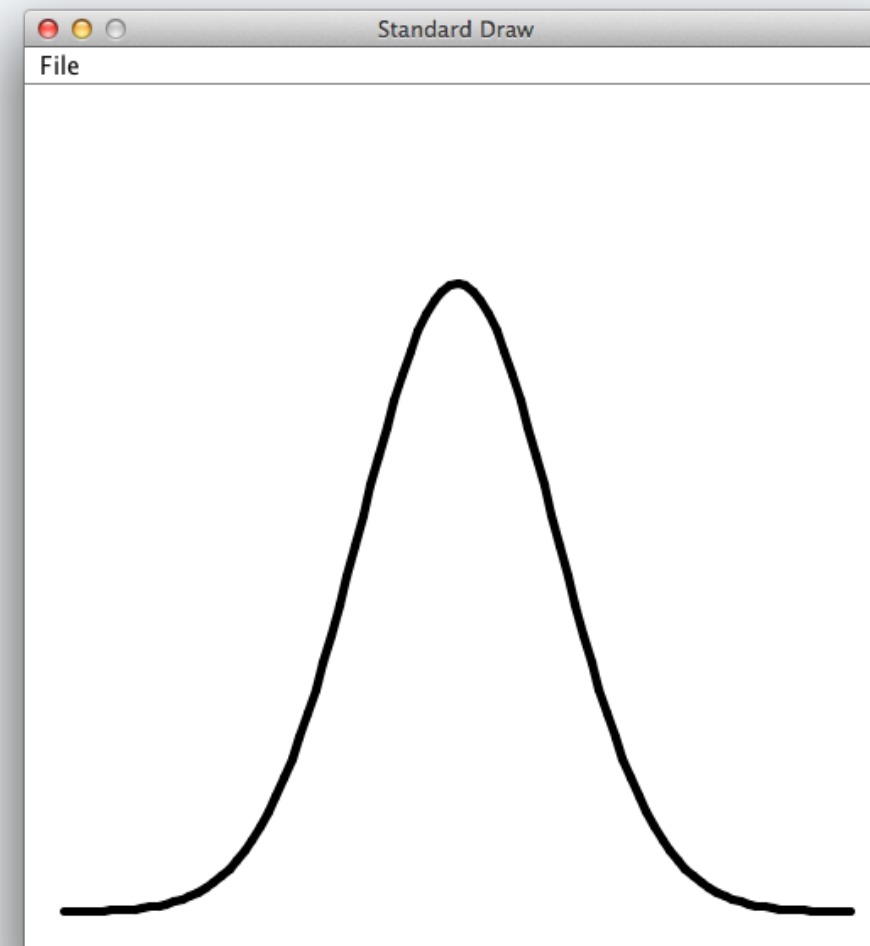
To use these methods in another program

- Put a copy of `Gaussian.java` in your working directory.
- Call `Gaussian.pdf()` or `Gaussian.cdf()` from any other module in that directory.

Learn to use your OS "classpath" mechanism if you find yourself with too many copies.

Example. Draw a plot of $\phi(x, 0, 1)$ in $(-4, 4)$

```
% java GaussianPlot 200
```



Libraries of functions provide an easy way for *any* user (you) to extend the Java system.

```
public class GaussianPlot
{
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        StdDraw.setXscale(-4.0, +4.0);
        StdDraw.setYscale(0, .5);
        StdDraw.setPenRadius(0.01);
        double[] x = new double[N+1];
        double[] y = new double[N+1];
        for (int i = 0; i <= N; i++)
        {
            x[i] = -4.0 + 8.0 * i / N;
            y[i] = Gaussian.pdf(x[i]);
        }
        for (int i = 0; i < N; i++)
            StdDraw.line(x[i], y[i], x[i+1], y[i+1]);
    }
}
```

COMPUTER SCIENCE

SEDGWICK / WAYNE

PART I: PROGRAMMING IN JAVA

Image sources

<http://www.intechopen.com/books/rheology-new-concepts-applications-and-methods/a-practical-review-of-microrheological-techniques>

<http://tweezpal.natan.si>

<http://www.albartlett.org/articles/art2000jan.html>

<http://laser.physics.sunysb.edu/~vwang/beam-propagation/>

<http://www.nature.com/neuro/journal/v14/n12/full/nn.2958.html>

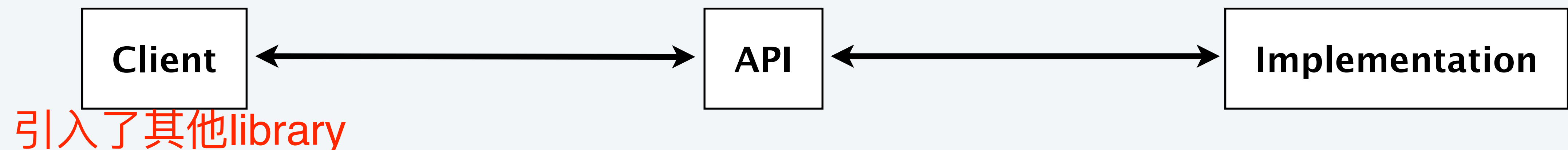
<http://www-cdf.fnal.gov/physics/new/top/2012/WHe19ifb/>

http://en.wikipedia.org/wiki/Deutsche_Mark#mediaviewer/File:DEU-10m-anv.jpg

5. Functions and Libraries

- Basic concepts
- Case study: Digital audio
- Application: Gaussian distribution
- **Modular programming**

Fundamental abstractions for modular programming



Client

Module that calls a library's methods.

```
public class GaussianPlot
{
    ...
    y[i] = Gaussian.pdf(x[i]);
    ...
}
```

Applications programming interface (API)

Defines signatures, describes methods.

public class Gaussian	
double pdf(double x)	<i>Gaussian probability density function</i>
double cdf(double x)	<i>Gaussian cumulative distribution function</i>

Implementation

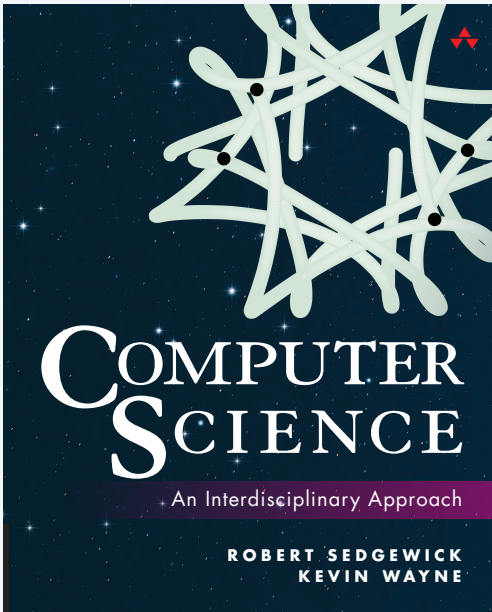
Module containing library's Java code.

```
public class Gaussian
{
    public static double pdf(double x)
    {
        double val = Math.exp(-x*x / 2);
        val /= Math.sqrt(2 * Math.PI);
        return val
    }
    ...
}
```

Example: StdRandom library

Developed for this course, but broadly useful

- Implement methods for generating random numbers of various types.
- Available for download at booksite (and included in introcs software).



API

public class StdRandom	
int uniform(int N)	<i>integer between 0 and N-1</i>
double uniform(double lo, double hi)	<i>real between lo and hi</i>
boolean bernoulli(double p)	<i>true with probability p</i>
double gaussian()	<i>normal with mean 0, stddev 1</i>
double gaussian(double m, double s)	<i>normal with mean m, stddev s</i>
int discrete(double[] a)	<i>i with probability a[i]</i>
void shuffle(double[] a)	<i>randomly shuffle the array a[]</i>

First step in developing a library: **Articulate the API!**

StdRandom details

Implementation

```
public class StdRandom
{
    public static double uniform(double a, double b)
    { return a + Math.random() * (b-a); }

    public static int uniform(int N)
    { return (int) (Math.random() * N); }

    public static boolean bernoulli(double p)
    { return Math.random() < p; }

    public static double gaussian()
        /* see Exercise 1.2.27 */

    public static double gaussian(double m, double s)
    { return m + (s * gaussian()); }

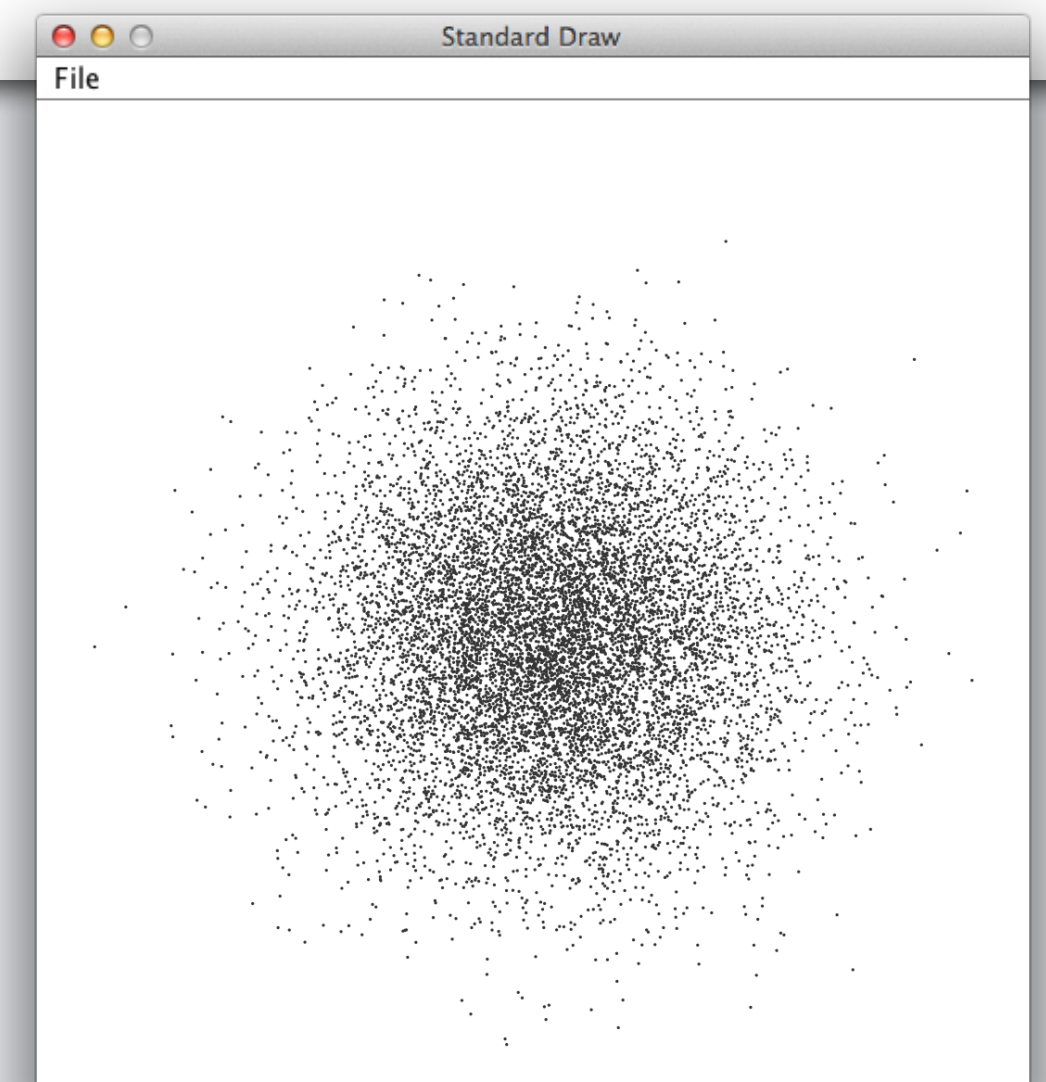
    ...
}
```

You *could* implement many of these methods,
but now you don't have to!

Typical client

```
public class RandomPoints
{
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 0; i < N; i++)
        {
            double x = StdRandom.gaussian(0.5, 0.2);
            double y = StdRandom.gaussian(0.5, 0.2);
            StdDraw.point(x, y);
        }
    }
}
```

```
% java RandomPoints 10000
```



Best practices

Small modules

- Separate and classify small tasks.
- Implement a layer of abstraction.

Independent development

- Code client *before* coding implementation.
- Anticipate needs of future clients.

Test clients

- Include `main()` test client in each module. ← run all code at least once!
- Do more extensive testing in a separate module.



```
public class StdRandom
{
    ...
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 0; i < N; i++) {
            StdOut.printf(" %2d " , uniform(100));
            StdOut.printf("%8.5f " , uniform(10.0, 99.0));
            StdOut.printf("%5b " , bernoulli(.5));
            StdOut.printf("%7.5f " , gaussian(9.0, .2));
            StdOut.println();
        }
    }
}
```

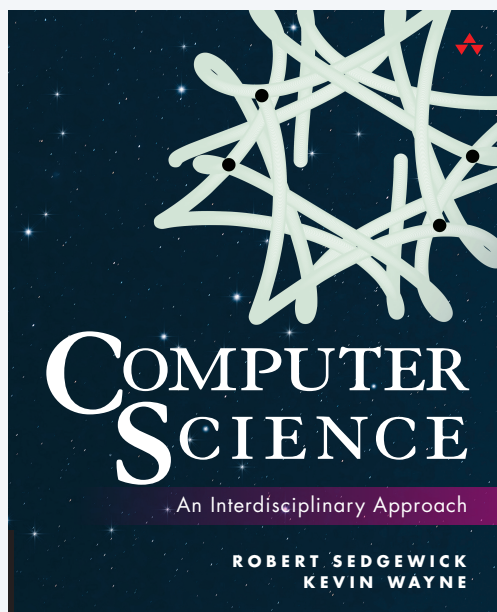
```
% java StdRandom 5
61 21.76541 true 9.30910
57 43.64327 false 9.42369
31 30.86201 true 9.06366
92 39.59314 true 9.00896
36 28.27256 false 8.66800
```

Example: StdStats library

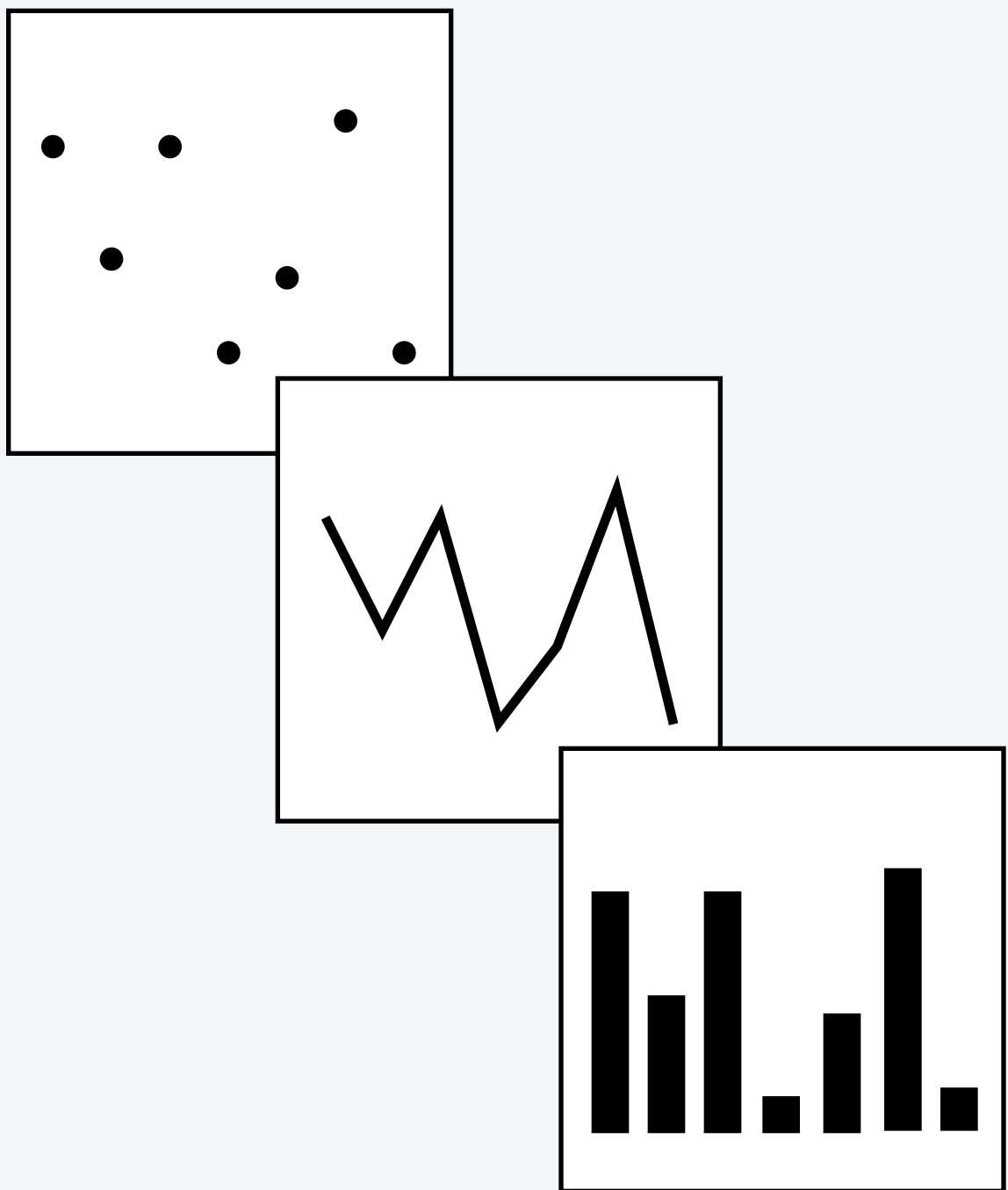
Developed for this course, but broadly useful

- Implement methods for computing statistics on arrays of real numbers.
- Available for download at booksite (and included in introcs software).

and plotting
on StdDraw



API	<code>public class StdStats</code>	
	<code>double max(double[] a)</code>	<i>largest value</i>
	<code>double min(double[] a)</code>	<i>smallest value</i>
	<code>double mean(double[] a)</code>	<i>average</i>
	<code>double var(double[] a)</code>	<i>sample variance</i>
	<code>double stddev(double[] a)</code>	<i>sample standard deviation</i>
	<code>double median(double[] a)</code>	<i>median</i>
	<code>void plotPoints(double[] a)</code>	<i>plot points at (i, a[i])</i>
	<code>void plotLines(double[] a)</code>	<i>plot lines connecting points at (i, a[i])</i>
	<code>void plotBars(double[] a)</code>	<i>plot bars to points at (i, a[i])</i>



Easy to implement, but **easier to use!**

← one reason to develop a library: clarify client code

Example of modular programming: StdStats, StdRandom, and Gaussian client

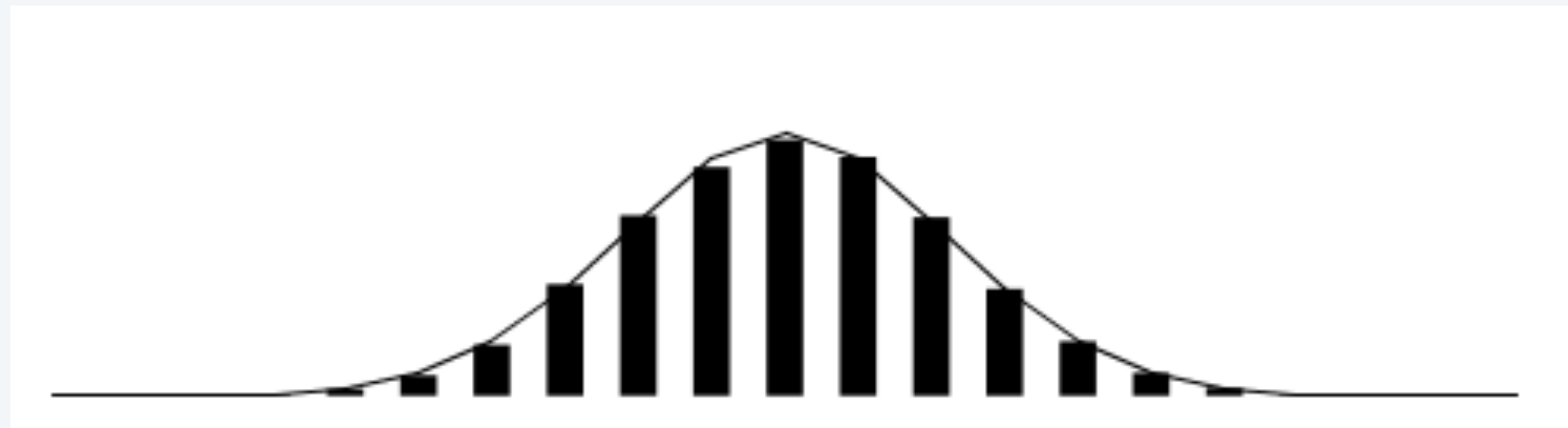
Experiment

- Flip N coins.
- How many heads?
- Prediction: Expect $N/2$.

```
public static int binomial(int N)
{
    int heads = 0;
    for (int j = 0; j < N; j++)
        if (StdRandom.bernoulli(0.5))
            heads++;
    return heads;
}
```

Prediction (more detailed)

- Run experiment *trials* times.
- How many heads?



Goal. Write a program to validate predictions.

Example of modular programming: Bernoulli trials

```
public class Bernoulli
{
    public static int binomial(int N)
    // See previous slide.

    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        int trials = Integer.parseInt(args[1]);

        int[] freq = new int[N+1];
        for (int t = 0; t < trials; t++)
            freq[binomial(N)]++;

        double[] normalized = new double[N+1];
        for (int i = 0; i <= N; i++)
            normalized[i] = (double) freq[i] / trials;
        StdStats.plotBars(normalized);

        double mean = N / 2.0;
        double stddev = Math.sqrt(N) / 2.0;
        double[] phi = new double[N+1];
        for (int i = 0; i <= N; i++)
            phi[i] = Gaussian.pdf(i, mean, stddev);
        StdStats.plotLines(phi);
    }
}
```

repeating for trials times

Bernoulli simulation

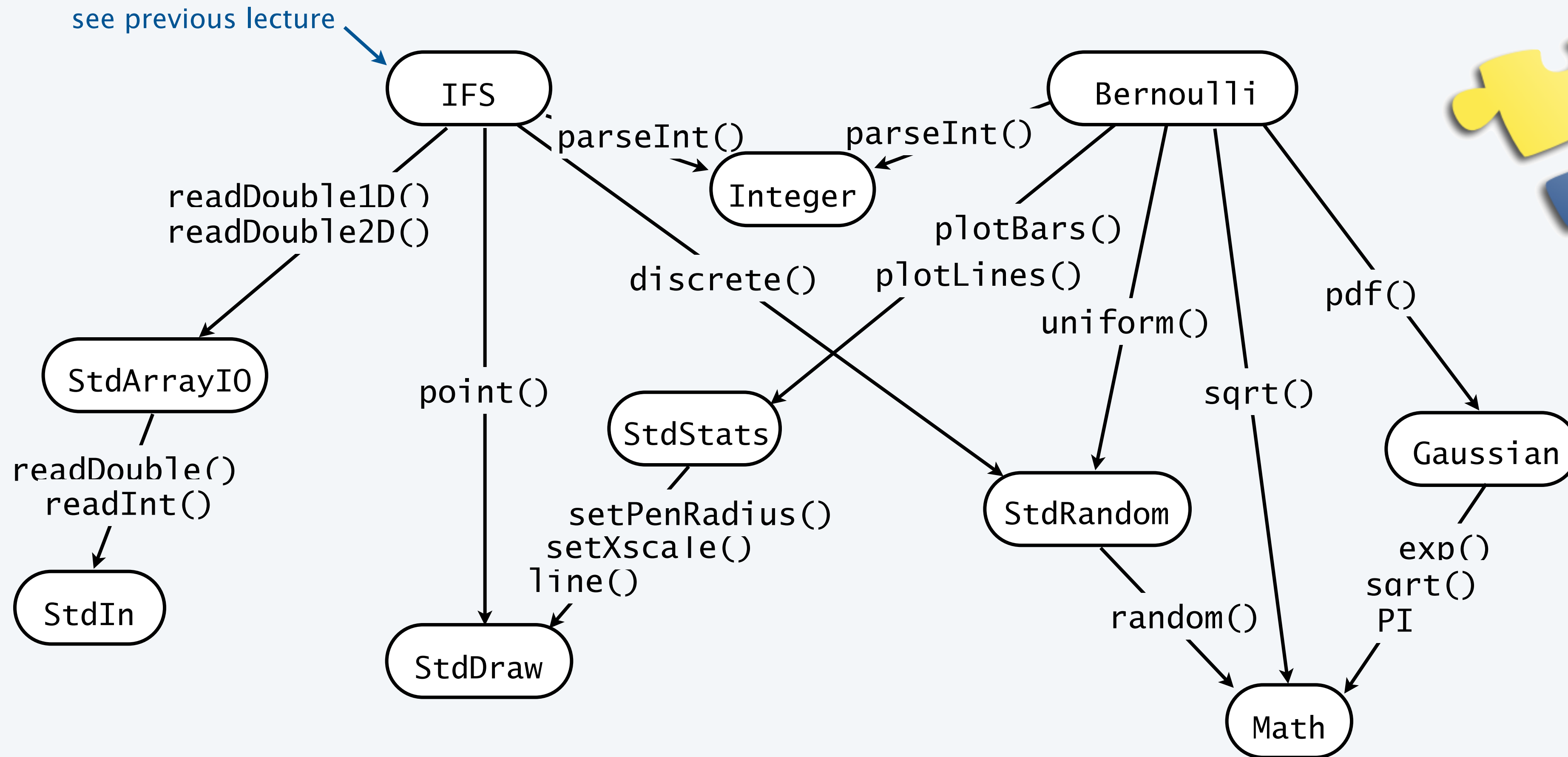
- Get command-line arguments (*trials* experiments of *N* flips).
- Run experiments. Keep track of frequency of occurrence of each return value.
- Normalize to between 0 and 1. Plot histogram.
- Plot theoretical curve.

```
% java Bernoulli 20 10000
```



Modular programming

enables development of complicated programs via simple independent modules.



Advantages. Code is easier to understand, debug, maintain, improve, and reuse.

Why modular programming?

Modular programming enables

- Independent development of small programs.
- Every programmer to develop and share layers of abstraction.
- Self-documenting code.

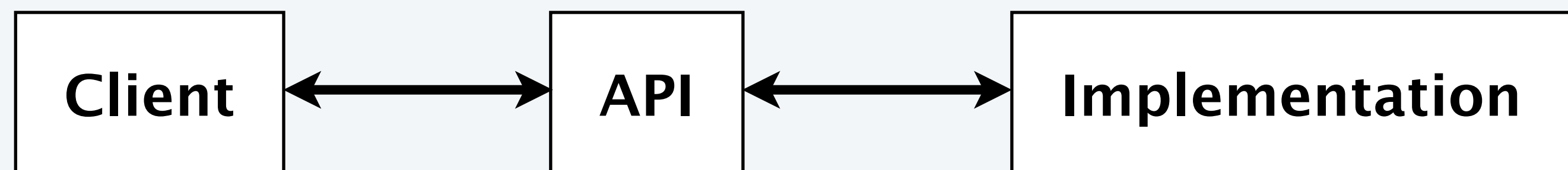


Fundamental characteristics

- Separation of client from implementation benefits all *future* clients.
- Contract between implementation and clients (API) benefits all *past* clients.

Challenges

- How to break task into independent modules?
- How to specify API?



COMPUTER SCIENCE

SEDGEWICK / WAYNE

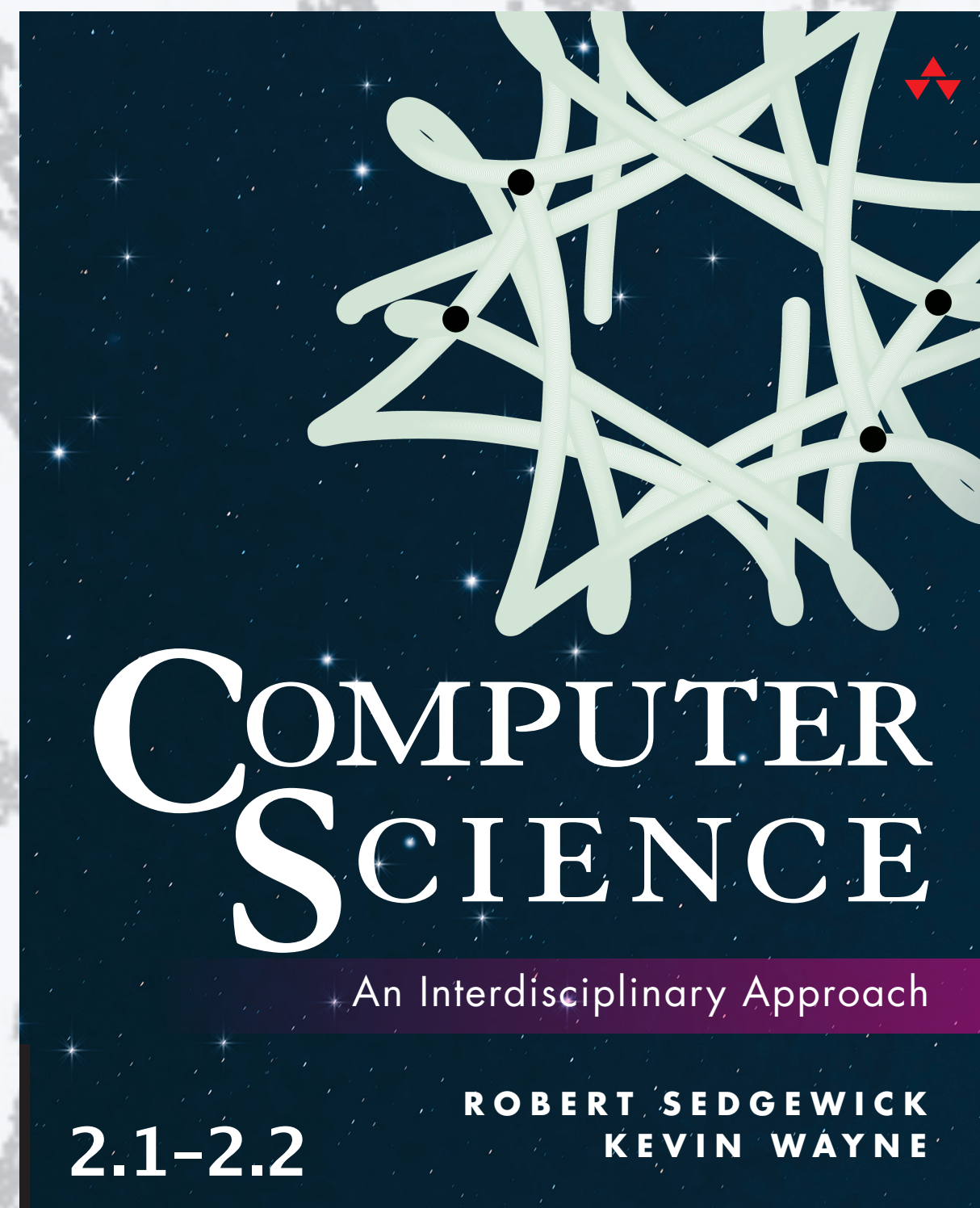
PART I: PROGRAMMING IN JAVA

Image sources

<http://xkcd.com/221/>

http://upload.wikimedia.org/wikipedia/commons/b/ba/Working_Together_Teamwork_Puzzle_Concept.jpg

http://upload.wikimedia.org/wikipedia/commons/e/ef/Ben_Jigsaw_Puzzle_Puzzle_Puzzle.png



5. Functions and Libraries